

Evaluación del momento de intensificación y fortalecimiento 02-03

Evaluación de la instancia «Evaluación del momento de intensificación y fortalecimiento 02-03» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-intensificaciones-02-03-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-intensificaciones-02-03-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación del momento de intensificación y fortalecimiento 02-03
Momento	Intensificación y fortalecimiento — encuentros 02-03
Carácter/Objetivo	Verificar saberes previos de lógica de programación, manejo de terminal y sistema de archivos, y reconocimiento de la plataforma de entregas GitHub, sin uso de Python.
Destinatarios	Todo el curso
Duración teórica	240 minutos (4 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-intensificaciones-02-03.md</code> · <code>evaluacion-intensificaciones-02-03-version-a.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (240 min)

Momento	Tiempo	Qué ocurre
Apertura y organización	10 min	El docente presenta la evaluación: ejercicios de saberes previos sin Python. Se explica el criterio Apto / No apto aún.
Parte 1 — Algoritmos en lenguaje natural	40 min	Ejercicios de lógica: secuencia, decisión y repetición expresados en lenguaje natural estructurado.
Parte 2 — Terminal y sistema de archivos	40 min	Ejercicios prácticos de navegación y creación de estructuras de carpetas usando comandos de terminal.
Parte 3 — Plataforma de entregas	30 min	Reconocimiento de GitHub: identificar repositorio, estructura de carpetas y mecanismo de entrega.
Cierre y verificación individual	20 min	Recorrido por los puestos y registro individual de objetivos logrados y pendientes.
Total	240 min	

3. Acuerdo pedagógico por grupo de condición

Grupo de intensificación (contenidos mínimos irrenunciables): ejercicios de lógica con algoritmos en lenguaje natural guiados paso a paso, práctica de terminal con comandos básicos (`cd` , `dir` / `ls` , `mkdir`), reconocimiento de la plataforma GitHub con navegación guiada del repositorio. **Actividad:** ejercicios escritos de secuencia, decisión y repetición; práctica de terminal; recorrido guiado por la plataforma. **Recursos:** guía impresa de comandos, computadora con VS Code y terminal.

Grupo de fortalecimiento: desafíos de lógica de mediana complejidad (búsqueda, ordenamiento conceptual, validación compuesta), comandos de terminal encadenados, y exploración autónoma de GitHub con bitácora personal. **Actividad:** problemas de lógica sin código, comandos combinados de terminal, navegación autónoma y documentación en bitácora. **Recursos:** consignas de desafío impresas, computadora con VS Code y terminal.

4. Desarrollo de los encuentros del momento

Encuentro 1 (120 min): apertura conjunta (10 min) — presentación de la evaluación y asignación de pistas. Bloque 1 (45 min): algoritmos en lenguaje natural — secuencia paso a paso (ordenar tres números, preparar una receta), decisión compuesta (decidir si abrir el kiosco según hora y día), repetición conceptual (contar boletos, sumar elementos de una lista en papel). Bloque 2 (45 min): terminal y archivos — comandos `pwd` , `cd` , `ls` / `dir` , `mkdir` , creación de la estructura de carpetas del curso. Cierre conjunto (20 min): puesta en común de soluciones.

Encuentro 2 (120 min): apertura conjunta (10 min) — repaso de terminal y lógica. Bloque 1 (45 min): consolidación de lógica y pseudocódigo — traducir diagramas de flujo del encuentro anterior a pseudocódigo estructurado, ejercicios de búsqueda conceptual. Bloque 2 (45 min): plataforma GitHub — ingreso a la URL del repositorio grupal, inicio de sesión, recorrido guiado por la estructura de carpetas (`tp-u1/` , `tp-u2/` , `trabajo-final/`), verificación de cuenta activa. Cierre conjunto (20 min): verificación individual y registro.

5. Criterios de calificación

Criterio: **Apto / No apto aún por objetivo mínimo.** No hay puntaje numérico.

Resultado	Condición
Apto	Resuelve al menos 2 de los 3 bloques de ejercicios (algoritmos, terminal, plataforma). Los algoritmos expresan correctamente secuencia y al menos una decisión o repetición. Crea estructuras de carpetas con comandos básicos. Ingresa a GitHub y navega el repositorio del curso.
No apto aún por objetivo mínimo	No resuelve al menos 2 bloques, o los algoritmos no reflejan comprensión de secuencia/decisión/repetición, o no logra navegar ni crear carpetas en terminal, o no puede acceder a GitHub. Se registran los objetivos pendientes y se deriva al siguiente momento de intensificación.

6. Condiciones de resolución de la prueba

Ejercicios de lógica: se entregan resueltos en papel o documento de texto (no se requiere código Python). Terminal: se evalúa en vivo con la computadora del alumno. Plataforma: se verifica que el alumno haya ingresado a GitHub con su cuenta y pueda señalar las carpetas correctas del repositorio. No se permite celular. Resolución individual.

7. Regla de equivalencia entre versiones

Estructura idéntica, mismos objetivos y requisitos, distintos datos de ejemplo. La versión B se genera desde la versión A mediante sustitución de tokens — no hay reglas que una versión tenga y otra no.

8. Mecánica de asignación de versiones

Se asignan por fila de la tabla del curso según orden de lista o según criterio de alternancia definido por el docente. Se registra la versión asignada en la planilla de calificaciones.

9. Devolución

Al cierre del encuentro: verificación individual y registro de objetivos logrados y pendientes. Los objetivos no alcanzados se retoman en los momentos de intensificación posteriores (encuentros 17-18).

Evaluación del momento 02-03 — Versión A

Dominio de esta versión: saberes previos (sin Python). Duración: 90 minutos. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-02-03.md](#).

Antes de empezar

- Tenés una computadora con VS Code, terminal y navegador web. No necesitás Python.
- Todos los ejercicios se resuelven en lenguaje natural estructurado (no código) o con comandos de terminal.
- El ejercicio de la plataforma se resuelve con el navegador web, ingresando a la URL del repositorio del curso que te indique el docente.
- No se permite celular. Trabajo individual.
- Al terminar, avisale al docente para que registre tus respuestas.

Objetivos de la prueba

- Expresar algoritmos simples en lenguaje natural estructurado (secuencia, decisión y repetición).
- Navegar por el sistema de archivos con comandos de terminal (`pwd`, `cd`, `ls / dir`, `mkdir`).
- Crear una estructura de carpetas desde la terminal.
- Ingresar a GitHub, iniciar sesión con tu cuenta y navegar el repositorio del curso.

Ejercicio 1 — Algoritmos en lenguaje natural (sin código)

1A — Secuencia de pasos (10 puntos conceptuales)

Escribí, paso a paso en lenguaje natural, el algoritmo para determinar si el kiosco escolar está abierto. Usá estas condiciones:
- El kiosco abre de lunes a viernes de 8:00 a 17:00. - Los sábados abre solo de 9:00 a 13:00. - Los domingos está cerrado. - Si llueve, el kiosco cierra 1 hora antes del horario normal.

Indicá claramente dónde hay una decisión (`si... entonces... sino...`) y dónde hay repetición si la hubiera.

1B — Repetición conceptual (10 puntos conceptuales)

Escribí, en lenguaje natural, el algoritmo para contar cuántas ventas se registraron en un día. La planilla tiene 15 anotaciones. Describí el proceso de recorrer la planilla y sumar uno por cada venta. Explicá qué se repite y cuándo termina la repetición.

Ejercicio 2 — Terminal y sistema de archivos

2A — Navegación (15 puntos)

Usando la terminal: 1. Mostrá la ruta del directorio actual con `pwd`. 2. Listá el contenido del directorio actual con `ls` (o `dir` en Windows). 3. Navegá a la carpeta `Documentos` (o equivalente en tu sistema) con `cd`.

Anotá en una hoja o documento de texto los comandos que usaste y qué mostró cada uno.

2B — Creación de estructura de carpetas (15 puntos)

Desde la terminal, creá la siguiente estructura de carpetas en tu escritorio (o en la ubicación que indique el docente):

```
LAP/  
  tp-u1/  
  tp-u2/  
  tp-u3/  
  trabajo-final/
```

Usá el comando `mkdir`. Anotá los comandos exactos que escribiste.

Ejercicio 3 — Plataforma de entregas

3A — Reconocimiento de GitHub (10 puntos)

1. Abrí el navegador web e ingresá la URL del repositorio del curso que te dé el docente.
2. Iniciá sesión en GitHub con tu cuenta. Si no tenés cuenta, el docente te va a asistir para crearla.
3. Una vez dentro del repositorio, señalá:
 - Dónde está la carpeta `tp-u1/`
 - Dónde está la carpeta `trabajo-final/`
4. Explicá con tus palabras para qué sirve cada carpeta.

3B — Entrega conceptual (10 puntos)

Respondé por escrito: - ¿Qué es un repositorio? - ¿Qué significa “hacer commit”? - ¿Qué carpeta del repositorio usarías para entregar tu primer trabajo de Python?

Al terminar

Avisá al docente. Él va a revisar tus respuestas del Ejercicio 1, verificar en vivo la terminal (Ejercicio 2) y el navegador (Ejercicio 3), y registrar los objetivos logrados en su lista de verificación.

Evaluación del momento 02-03 — Versión A

Dominio de esta versión: saberes previos (sin Python). Duración: 90 minutos. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-02-03.md](#).

Antes de empezar

- Tenés una computadora con VS Code, terminal y navegador web. No necesitás Python.
- Todos los ejercicios se resuelven en lenguaje natural estructurado (no código) o con comandos de terminal.
- El ejercicio de la plataforma se resuelve con el navegador web, ingresando a la URL del repositorio del curso que te indique el docente.
- No se permite celular. Trabajo individual.
- Al terminar, avísale al docente para que registre tus respuestas.

Objetivos de la prueba

- Expresar algoritmos simples en lenguaje natural estructurado (secuencia, decisión y repetición).
- Navegar por el sistema de archivos con comandos de terminal (`pwd`, `cd`, `ls / dir`, `mkdir`).
- Crear una estructura de carpetas desde la terminal.
- Ingresar a GitHub, iniciar sesión con tu cuenta y navegar el repositorio del curso.

Ejercicio 1 — Algoritmos en lenguaje natural (sin código)

1A — Secuencia de pasos (10 puntos conceptuales)

Escribí, paso a paso en lenguaje natural, el algoritmo para determinar si la biblioteca escolar está abierta. Usá estas condiciones: - La biblioteca abre de lunes a viernes de 8:00 a 17:00. - Los sábados abre solo de 9:00 a 13:00. - Los domingos está cerrado. - Si llueve, la biblioteca cierra 1 hora antes del horario normal.

Indicá claramente dónde hay una decisión (`si... entonces... sino...`) y dónde hay repetición si la hubiera.

1B — Repetición conceptual (10 puntos conceptuales)

Escribí, en lenguaje natural, el algoritmo para contar cuántas ventas se registraron en un día. La planilla tiene 15 anotaciones. Describí el proceso de recorrer la planilla y sumar uno por cada préstamo. Explicá qué se repite y cuándo termina la repetición.

Ejercicio 2 — Terminal y sistema de archivos

2A — Navegación (15 puntos)

Usando la terminal: 1. Mostrá la ruta del directorio actual con `pwd`. 2. Listá el contenido del directorio actual con `ls` (o `dir` en Windows). 3. Navegá a la carpeta `Documentos` (o equivalente en tu sistema) con `cd`.

Anotá en una hoja o documento de texto los comandos que usaste y qué mostró cada uno.

2B — Creación de estructura de carpetas (15 puntos)

Desde la terminal, creá la siguiente estructura de carpetas en tu escritorio (o en la ubicación que indique el docente):

```
LAP/  
  tp-u1/  
  tp-u2/  
  tp-u3/  
  trabajo-final/
```

Usá el comando `mkdir`. Anotá los comandos exactos que escribiste.

Ejercicio 3 — Plataforma de entregas

3A — Reconocimiento de GitHub (10 puntos)

1. Abrí el navegador web e ingresá la URL del repositorio del curso que te dé el docente.
2. Iniciá sesión en GitHub con tu cuenta. Si no tenés cuenta, el docente te va a asistir para crearla.
3. Una vez dentro del repositorio, señalá:
 - Dónde está la carpeta `tp-u1/`
 - Dónde está la carpeta `trabajo-final/`
4. Explicá con tus palabras para qué sirve cada carpeta.

3B — Entrega conceptual (10 puntos)

Respondé por escrito: - ¿Qué es un repositorio? - ¿Qué significa “hacer commit”? - ¿Qué carpeta del repositorio usarías para entregar tu primer trabajo de Python?

Al terminar

Avisá al docente. Él va a revisar tus respuestas del Ejercicio 1, verificar en vivo la terminal (Ejercicio 2) y el navegador (Ejercicio 3), y registrar los objetivos logrados en su lista de verificación.

Evaluación del momento de intensificación y fortalecimiento 17-18

Evaluación de la instancia «Evaluación del momento de intensificación y fortalecimiento 17-18» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-intensificaciones-17-18-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-intensificaciones-17-18-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación del momento de intensificación y fortalecimiento 17-18
Momento	Intensificación y fortalecimiento — encuentros 17-18
Carácter/Objetivo	Evaluar los núcleos de las Unidades 1 y 2: entrada/salida, tipos, condicionales, bucles, listas, diccionarios y funciones, mediante un programa corto de consola.
Destinatarios	Todo el curso
Duración teórica	240 minutos (4 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-intensificaciones-17-18.md</code> · <code>evaluacion-intensificaciones-17-18-version-a.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (240 min)

Momento	Tiempo	Qué ocurre
Apertura y organización	10 min	Presentación de la consigna, criterios y organización de pistas.
Bloque 1 — Programa corto: entrada y condicionales	45 min	Ejercicio guiado: programa que pida datos del dominio (kiosco + club) y aplique condicionales para decidir el flujo.
Bloque 2 — Bucle y lista con funciones	45 min	Completar el programa con un bucle <code>while</code> para menú, una lista para almacenar datos y funciones para organizar el código.
Bloque 3 — Cierre Git	20 min	Commit y push del programa en el repo grupal.
Cierre y verificación individual	20 min	Recorrido por los puestos, verificación de objetivos y registro.
Total	240 min	

3. Acuerdo pedagógico por grupo de condición

Grupo de intensificación (contenidos mínimos irrenunciables): escribir un programa corto de consola que use `print()` /

`input()` , condicionales `if / elif / else` , bucle `while` con menú, una lista para almacenar datos y al menos una función con `def` y `return` . Actividad: ejercicios guiados paso a paso con corrección inmediata y cierre con commit al repo. Recursos: VS Code, terminal, repo grupal clonado, guía impresa de núcleos U1 y U2.

Grupo de fortalecimiento: extender el programa base con funcionalidades adicionales (ordenamiento, búsqueda por múltiples criterios, uso de diccionarios anidados, funciones con argumentos por nombre y valores default). Actividad: desafíos autónomos de extensión sobre el mismo programa base. Recursos: VS Code, terminal, repo grupal, consignas de desafío impresas.

4. Desarrollo de los encuentros del momento

Encuentro 17 (120 min): apertura conjunta (10 min) — mapa conceptual de U1+U2, asignación de pistas. Bloque 1 (45 min): entrada, tipos y condicionales sobre el dominio del kiosco escolar (productos, precios, stock, descuento por cliente). Bloque 2 (45 min): bucles, listas y funciones sobre el dominio del club de barrio (socios, cuotas, vencimientos, miembros activos). Cierre conjunto (20 min): puesta en común y verificación de avances.

Encuentro 18 (120 min): apertura conjunta (10 min) — repaso y objetivos del día. Bloque 1 (45 min): diccionarios y funciones — modelar socios como diccionarios dentro de una lista, funciones de búsqueda y filtrado. Bloque 2 (45 min): bloque principal (`if __name__`), refactorización del programa anterior y cierre con commit y push. Cierre conjunto (20 min): verificación individual y registro de objetivos.

5. Criterios de calificación

Criterio: **Apto / No apto aún por objetivo mínimo**. No hay puntaje numérico.

Resultado	Condición
Apto	El programa compila y ejecuta. Usa <code>input()</code> / <code>print()</code> con tipos correctos, condicionales (<code>if / elif / else</code>), bucle <code>while</code> con menú, una lista para almacenar datos y al menos una función con <code>def</code> y <code>return</code> . Tiene bloque principal <code>if __name__ == "__main__"</code> . Hay commit en el repo.
No apto aún por objetivo mínimo	El programa no compila, no ejecuta, o falta alguno de los elementos anteriores (sin condicionales, sin bucle, sin funciones, sin lista, sin bloque principal, sin commit). Se registran los objetivos pendientes y se deriva al proyecto puente (encuentros 19-20).

6. Condiciones de resolución de la prueba

Resolución individual con computadora, VS Code y terminal. No se permite celular. El alumno escribe un único archivo `.py` . Debe cerrar con commit y push al repo grupal. Se permite consultar la guía impresa de núcleos U1 y U2 y los ejemplos resueltos de los TP-U1 y TP-U2.

7. Regla de equivalencia entre versiones

Misma estructura, mismos objetivos y requisitos, distinto dominio (kiosco + club vs. biblioteca + taller). La versión B se genera desde la versión A mediante sustitución de tokens de la tabla de dominio — no hay reglas que una versión tenga y otra no.

8. Mecánica de asignación de versiones

Se asignan por fila de la tabla del curso según orden de lista. Se registra la versión asignada en la planilla de calificaciones.

9. Devolución

Al cierre del encuentro 18: verificación individual y registro de objetivos logrados y pendientes. Los objetivos no alcanzados se retoman en el proyecto puente (encuentros 19-20) o en la intensificación de Unidades 3 y 4 (encuentros 34-35) según el criterio del docente.

Evaluación del momento 17-18 — Versión A

Dominio de esta versión: kiosco escolar (U1) + club de barrio (U2). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-17-18.md](#).

Antes de empezar

- Crea un archivo `kiosco_club.py` en la carpeta `tp-u1/` o `tp-u2/` de tu repo grupal.
- Escribí el programa en un solo archivo `.py`. Definí las funciones primero, el bloque de ejecución al final con `if __name__ == "__main__":`.
- Usá `input()` para leer datos y `print()` con f-strings para mostrar resultados.
- No se permite celular. Trabajo individual.
- Al terminar, hace commit y push al repo grupal.

Objetivos de la prueba

- Usar `print()` / `input()` con conversión de tipos (`int()`, `float()`).
- Aplicar condicionales `if/elif/else` para decidir el flujo del programa.
- Usar un bucle `while` para mantener un menú activo hasta que el usuario decida salir.
- Almacenar datos en una lista.
- Definir y usar al menos una función con `def`, parámetros y `return`.

Consigna — Programa corto de kiosco escolar con registro de clientes

Escribí un programa que gestione las ventas del **kiosco escolar**. El programa debe comenzar con una lista vacía y un menú que se repite hasta que el usuario elija salir.

Datos de ejemplo del dominio

- **articulos:** ["Caramelos", "Galletitas", "Jugo"]
- **precio:** 50, 120, 80 (en pesos)
- **stock:** 100, 50, 30 (unidades disponibles)
- **cliente:** nombre de quien compra

Requisitos del programa

1. **Menú principal** con `while True` y opciones:
 - 1: Registrar venta (opción `registerSale`)
 - 2: Mostrar articulos con stock
 - 3: Mostrar clientes registrados
 - 4: Calcular total de ventas del día (`saleTotal`)
 - 5: Salir
2. **Opción 1 — Registrar venta** (`registerSale`)

- Pedir nombre del **cliente**.
- Pedir nombre del **producto** y cantidad.
- Verificar que haya **stock** suficiente. Si no hay, mostrar “Stock insuficiente”.
- Si hay stock, restar la cantidad del **stock** y calcular el total (`itemPrice` × cantidad).
- Aplicar un **descuento** del 10% si el total supera los \$200.
- Guardar la venta en una lista (cada venta como un diccionario con los datos).

3. Opción 2 — Mostrar artículos

- Recorrer la lista de **artículos** con un bucle `for` y mostrar nombre, **precio** y **stock** disponible.

4. Opción 3 — Mostrar clientes registrados

- Mostrar todos los **clientes** que compraron hoy, sin repetir.

5. Opción 4 — Calcular total de ventas (`saleTotal`)

- Sumar el total de todas las ventas registradas y mostrarlo con f-string formateado.

Función requerida

Definí al menos una función `registerSale(articulos, stock, ventas, descuento, cliente, itemPrice, cantidad)` que reciba los datos necesarios, realice la venta si hay stock y devuelva la venta registrada o un mensaje de error.

Formato de salida esperado

```
=== KIOSCO ESCOLAR ===
1. Registrar venta
2. Mostrar articulos
3. Mostrar clientes
4. Calcular total de ventas
5. Salir
Opción: 1
Nombre del cliente: Ana
Producto: Caramelos
Cantidad: 3
Venta registrada: Ana compró 3 Caramelos por $150.00
```

Al terminar

Commit con mensaje `"feat: evaluacion 17-18 kiosco"` y push al repo grupal. Avisá al docente para que verifique.

Evaluación del momento 17-18 — Versión A

Dominio de esta versión: biblioteca escolar (U1) + taller de barrio (U2). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-17-18.md](#).

Antes de empezar

- Crea un archivo `biblioteca_taller.py` en la carpeta `tp-u1/` o `tp-u2/` de tu repo grupal.
- Escribí el programa en un solo archivo `.py`. Definí las funciones primero, el bloque de ejecución al final con `if __name__ == "__main__":`.
- Usá `input()` para leer datos y `print()` con f-strings para mostrar resultados.
- No se permite celular. Trabajo individual.
- Al terminar, hace commit y push al repo grupal.

Objetivos de la prueba

- Usar `print()` / `input()` con conversión de tipos (`int()`, `float()`).
- Aplicar condicionales `if/elif/else` para decidir el flujo del programa.
- Usar un bucle `while` para mantener un menú activo hasta que el usuario decida salir.
- Almacenar datos en una lista.
- Definir y usar al menos una función con `def`, parámetros y `return`.

Consigna — Programa corto de biblioteca escolar con registro de clientes

Escribí un programa que gestione las ventas del **biblioteca escolar**. El programa debe comenzar con una lista vacía y un menú que se repite hasta que el usuario elija salir.

Datos de ejemplo del dominio

- **libros:** ["Caramelos", "Galletitas", "Jugo"]
- **tarifa:** 50, 120, 80 (en pesos)
- **ejemplares:** 100, 50, 30 (unidades disponibles)
- **lector:** nombre de quien compra

Requisitos del programa

1. **Menú principal** con `while True` y opciones:
 - 1: Registrar préstamo (opción `registerLoan`)
 - 2: Mostrar libros con ejemplares
 - 3: Mostrar clientes registrados
 - 4: Calcular total de ventas del día (`loanTotal`)
 - 5: Salir
2. **Opción 1 — Registrar préstamo** (`registerLoan`)

- Pedir nombre del **lector**.
- Pedir nombre del **película** y cantidad.
- Verificar que haya **ejemplares** suficiente. Si no hay, mostrar “Ejemplares insuficiente”.
- Si hay ejemplares, restar la cantidad del **ejemplares** y calcular el total (`bookPrice` × cantidad).
- Aplicar un **recargo** del 10% si el total supera los \$200.
- Guardar la préstamo en una lista (cada préstamo como un diccionario con los datos).

3. Opción 2 — Mostrar libros

- Recorrer la lista de **libros** con un bucle `for` y mostrar nombre, **tarifa** y **ejemplares** disponible.

4. Opción 3 — Mostrar clientes registrados

- Mostrar todos los **clientes** que compraron hoy, sin repetir.

5. Opción 4 — Calcular total de ventas (`loanTotal`)

- Sumar el total de todas las ventas registradas y mostrarlo con f-string formateado.

Función requerida

Definí al menos una función `registerLoan(libros, ejemplares, ventas, recargo, lector, bookPrice, cantidad)` que reciba los datos necesarios, realice la préstamo si hay ejemplares y devuelva la préstamo registrada o un mensaje de error.

Formato de salida esperado

```
=== BIBLIOTECA ESCOLAR ===
1. Registrar préstamo
2. Mostrar libros
3. Mostrar clientes
4. Calcular total de ventas
5. Salir
Opción: 1
Nombre del lector: Ana
Película: Caramelos
Cantidad: 3
Préstamo registrada: Ana compró 3 Caramelos por $150.00
```

Al terminar

Commit con mensaje `"feat: evaluacion 17-18 biblioteca"` y push al repo grupal. Avisá al docente para que verifique.

Evaluación del momento de intensificación y fortalecimiento 19-20

Evaluación de la instancia «Evaluación del momento de intensificación y fortalecimiento 19-20» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-intensificaciones-19-20-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-intensificaciones-19-20-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación del momento de intensificación y fortalecimiento 19-20
Momento	Intensificación y fortalecimiento — encuentros 19-20 (proyecto puente)
Carácter/Objetivo	Evaluar la integración de las Unidades 1 y 2 mediante un proyecto integrador de consola con rúbrica de 100 puntos. Función recuperatoria: los estudiantes que no alcanzaron los objetivos mínimos en las unidades 1 o 2 tienen aquí una instancia de aprobación.
Destinatarios	Todo el curso
Duración teórica	240 minutos (4 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-intensificaciones-19-20.md</code> · <code>evaluacion-intensificaciones-19-20-version-a.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (240 min)

Momento	Tiempo	Qué ocurre
Apertura y consigna	10 min	Presentación del proyecto puente y la rúbrica de 100 puntos.
Bloque 1 — Estructura base y opciones 1 y 2	45 min	Implementar el esqueleto del programa, menú principal, opción de agregar y opción de listar.
Bloque 2 — Opciones 3, 4 y 5	45 min	Completar búsqueda, filtrado de activos y filtrado de vencidos.
Bloque 3 — Bloque principal, pulido y entrega	45 min	Refactorizar con <code>if __name__</code> , validación mínima, comentarios y commit final.
Cierre con defensa	20 min	Defensa individual: cada estudiante explica su programa, estructura de datos y una función compleja.
Total	240 min	

3. Acuerdo pedagógico por grupo de condición

Única pista para todo el curso. Todos los estudiantes reciben la misma consigna y trabajan con la misma rúbrica de 100 puntos. No hay pistas diferenciadas porque el proyecto puente es recuperatorio: quienes ya aprobaron U1 y U2 completan el proyecto como consolidación; quienes no aprobaron lo usan como instancia de recuperación.

Contenidos: proyecto integrador que articula entrada/salida (`print`, `input`), condicionales (`if/elif/else`), bucles (`while`, `for`), listas, diccionarios y funciones (`def`, parámetros, `return`). Dominio: sistema de gestión de un club de barrio (socios, cuotas, vencimientos).

Actividad/metodología: construcción progresiva del programa en dos encuentros. Encuentro 19: consigna, planificación grupal, opciones 1 y 2. Encuentro 20: opciones 3-5, refactorización, defensa y entrega.

Recursos: VS Code, terminal, repo grupal clonado, consigna impresa con rúbrica de 100 puntos, ejemplos resueltos de U1 y U2.

4. Desarrollo de los encuentros del momento

Encuentro 19 (120 min): apertura (10 min) — lectura de la consigna, explicación de la rúbrica. Bloque 1 (45 min): planificación en parejas de la estructura del programa (funciones necesarias, estructura de datos, flujo del menú) y puesta en común. Bloque 2 (45 min): implementación de opción 1 (agregar socio con diccionario) y opción 2 (listar socios con `for`). Cierre (20 min): verificación de avances, commit con mensaje `"feat: agregar socio y listar socios"`.

Encuentro 20 (120 min): apertura (10 min) — repaso y objetivos del día. Bloque 1 (45 min): opción 3 (buscar socio por nombre con función `findMember`), opción 4 (socios activos), opción 5 (socios vencidos). Bloque 2 (45 min): refactorización con `if __name__`, validación con `try/except` básico, comentarios. Cierre con defensa (20 min): recorrido por los puestos, defensa individual y registro de puntajes en la rúbrica.

5. Criterios de calificación — rúbrica (100 puntos)

Dimensión	Puntaje	Indicadores
Entrada y condicionales	20 pts	Usa <code>input()</code> y <code>int()/float()</code> correctamente. Aplica <code>if/elif/else</code> para decidir flujo del menú y validar datos.
Bucles	15 pts	Usa <code>while</code> para el menú principal (hasta que el usuario elija salir). Usa <code>for</code> para recorrer listas y diccionarios.
Listas y diccionarios	25 pts	Almacena socios como diccionarios dentro de una lista. Opera sobre la lista (agregar, eliminar, buscar, listar).
Funciones y bloque principal	20 pts	Define funciones con <code>def</code> , parámetros y <code>return</code> . Organiza el programa con <code>if __name__ == "__main__"</code> .
Calidad del código	10 pts	Nombres descriptivos, comentarios útiles, código sin líneas muertas, entrada de datos validada mínimamente.
Entrega y defensa	10 pts	Repositorio GitHub actualizado con commit del proyecto. Breve explicación oral del programa y las decisiones tomadas.

Aprobado: ≥ 60 puntos. **Destacado:** ≥ 85 puntos. **No aprobado:** < 60 puntos.

6. Condiciones de resolución de la prueba

Resolución individual con computadora, VS Code y terminal. No se permite celular. El alumno escribe un único archivo `.py`. Debe cerrar con commit y push al repo grupal. Se permite consultar la guía impresa de la consigna y los ejemplos resueltos de U1 y U2.

7. Regla de equivalencia entre versiones

Misma estructura, mismos objetivos, mismo puntaje por dimensión, distinto dominio (club vs. taller). La versión B se genera desde la versión A mediante sustitución de tokens de la tabla de dominio.

8. Mecánica de asignación de versiones

Se asignan por fila de la tabla del curso según orden de lista. Se registra la versión asignada en la planilla de calificaciones.

9. Devolución

Al finalizar el encuentro 20: entrega del puntaje parcial por dimensión de la rúbrica y el resultado (aprobado / no aprobado). Los estudiantes no aprobados pasan al grupo de intensificación en los momentos siguientes (encuentros 34-35 o diciembre).

Evaluación del momento 19-20 — Versión A

Dominio de esta versión: club de barrio (socios, cuotas, vencimientos). Duración: 90 minutos por encuentro. Puntaje total: 100 puntos según rúbrica. Resolución individual, con computadora, sin celular. Las condiciones completas están en `evaluacion-intensificaciones-19-20.md`.

Antes de empezar

- Crea un archivo `club.py` en la carpeta `tp-u2/` de tu repo grupal.
- Escribí el programa en un solo archivo `.py`. Definí las funciones primero, el bloque de ejecución al final con `if __name__ == "__main__":`.
- No uses archivos ni base de datos: todo se almacena en listas y diccionarios en memoria.
- No se permite celular. Trabajo individual.
- Al terminar cada encuentro, hace commit y push.

Objetivos de la prueba

- Integrar entrada/salida, condicionales, bucles, listas, diccionarios y funciones en un solo programa.
- Organizar el código con funciones y bloque principal.
- Validar entrada de datos mínimamente.
- Entregar en GitHub con commit y README.

Consigna — Sistema de gestión de socios de un club de barrio

Escribí un programa que gestione los **socios** de un **club** de barrio. Cada **socio** es un diccionario con las claves: - `"nombre"`: string - `"edad"` (`memberAge`): entero - `"cuota"` (`clubFee`): float (monto de la cuota mensual) - `"vencimiento"`: string (mes de vencimiento, ej: "marzo")

Los socios se almacenan en una lista de diccionarios.

Menú principal (con `while True`)

```
=== SISTEMA DE SOCIOS DEL CLUB ===
1. Agregar socio
2. Listar todos los socios
3. Buscar socio por nombre
4. Mostrar socios activos (cuota al día)
5. Mostrar socios con cuota vencida
6. Salir
```

Funciones requeridas (mínimo)

Función	Descripción
<code>agregar_socio(memberList)</code>	Pide nombre, edad, cuota y vencimiento. Crea un diccionario y lo agrega a la lista. Valida que edad y cuota sean números (con <code>try / except</code>).
<code>listar_socios(memberList)</code>	Recorre la lista con <code>for</code> y muestra cada socio con formato.
<code>buscar_socio(memberList, nombre)</code> (<code>findMember</code>)	Busca por nombre en la lista. Devuelve el diccionario o "Socio no encontrado".
<code>socios_activos(memberList, mes_actual)</code> (<code>activeMembers</code>)	Filtra los socios cuyo vencimiento es igual o posterior al mes actual. Los devuelve en una nueva lista.
<code>socios_vencidos(memberList, mes_actual)</code>	Filtra los socios con vencimiento anterior al mes actual.

Rúbrica de 100 puntos

Dimensión	Puntaje	Indicadores
Entrada y condicionales	20 pts	Usa <code>input()</code> y <code>int()</code> / <code>float()</code> correctamente. Aplica <code>if / elif / else</code> para decidir flujo del menú y validar datos.
Bucles	15 pts	Usa <code>while</code> para el menú principal. Usa <code>for</code> para recorrer listas y diccionarios.
Listas y diccionarios	25 pts	Almacena socios como diccionarios dentro de una lista. Opera sobre la lista (agregar, eliminar, buscar, listar).
Funciones y bloque principal	20 pts	Define funciones con <code>def</code> , parámetros y <code>return</code> . Organiza el programa con <code>if __name__ == "__main__"</code> .
Calidad del código	10 pts	Nombres descriptivos, comentarios útiles, código sin líneas muertas, entrada validada mínimamente.
Entrega y defensa	10 pts	Repositorio GitHub actualizado con commit del proyecto. Breve explicación oral del programa y las decisiones tomadas.

Aprobado: ≥ 60 puntos. **Destacado:** ≥ 85 puntos.

Formato de salida esperado

```

=== SISTEMA DE SOCIOS DEL CLUB ===
1. Agregar socio
2. Listar todos los socios
3. Buscar socio por nombre
4. Mostrar socios activos
5. Mostrar socios con cuota vencida
6. Salir
Opción: 1
Nombre: Carlos
Edad: 34
Cuota: 1500
Vencimiento: mayo
Socio agregado correctamente.

```

Al terminar

Commit final con mensaje `"feat: proyecto puente club completo"` y push al repo grupal. Prepara una breve explicación oral (2 minutos) sobre qué hace tu programa, cómo elegiste las estructuras de datos y qué función te costó más.

Evaluación del momento 19-20 — Versión A

Dominio de esta versión: taller de barrio (herramientas, cuotas, vencimientos). Duración: 90 minutos por encuentro. Puntaje total: 100 puntos según rúbrica. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-19-20.md](#).

Antes de empezar

- Crea un archivo `taller.py` en la carpeta `tp-u2/` de tu repo grupal.
- Escribí el programa en un solo archivo `.py`. Definí las funciones primero, el bloque de ejecución al final con `if __name__ == "__main__":`.
- No uses archivos ni base de datos: todo se almacena en listas y diccionarios en memoria.
- No se permite celular. Trabajo individual.
- Al terminar cada encuentro, hace commit y push.

Objetivos de la prueba

- Integrar entrada/salida, condicionales, bucles, listas, diccionarios y funciones en un solo programa.
- Organizar el código con funciones y bloque principal.
- Validar entrada de datos mínimamente.
- Entregar en GitHub con commit y README.

Consigna — Sistema de gestión de herramientas de un taller de barrio

Escribí un programa que gestione los **herramientas** de un **taller** de barrio. Cada **herramienta** es un diccionario con las claves: - `"nombre"`: string - `"edad"` (`toolCondition`): entero - `"depósito"` (`toolDeposit`): float (monto de la depósito mensual) - `"devolución"`: string (mes de devolución, ej: "marzo")

Los herramientas se almacenan en una lista de diccionarios.

Menú principal (con `while True`)

```
=== SISTEMA DE HERRAMIENTAS DEL TALLER ===
1. Agregar herramienta
2. Listar todos los herramientas
3. Buscar herramienta por nombre
4. Mostrar herramientas activos (depósito al día)
5. Mostrar herramientas con depósito vencida
6. Salir
```

Funciones requeridas (mínimo)

Función	Descripción
<code>agregar_socio(toolList)</code>	Pide nombre, edad, depósito y devolución. Crea un diccionario y lo agrega a la lista. Valida que edad y depósito sean números (con <code>try / except</code>).
<code>listar_socios(toolList)</code>	Recorre la lista con <code>for</code> y muestra cada herramienta con formato.
<code>buscar_socio(toolList, nombre) (findTool)</code>	Busca por nombre en la lista. Devuelve el diccionario o “Herramienta no encontrado”.
<code>socios_activos(toolList, mes_actual)</code> <code>(availableTools)</code>	Filtra los herramientas cuyo devolución es igual o posterior al mes actual. Los devuelve en una nueva lista.
<code>socios_vencidos(toolList, mes_actual)</code>	Filtra los herramientas con devolución anterior al mes actual.

Rúbrica de 100 puntos

Dimensión	Puntaje	Indicadores
Entrada y condicionales	20 pts	Usa <code>input()</code> y <code>int()</code> / <code>float()</code> correctamente. Aplica <code>if / elif / else</code> para decidir flujo del menú y validar datos.
Bucles	15 pts	Usa <code>while</code> para el menú principal. Usa <code>for</code> para recorrer listas y diccionarios.
Listas y diccionarios	25 pts	Almacena herramientas como diccionarios dentro de una lista. Opera sobre la lista (agregar, eliminar, buscar, listar).
Funciones y bloque principal	20 pts	Define funciones con <code>def</code> , parámetros y <code>return</code> . Organiza el programa con <code>if __name__ == "__main__"</code> .
Calidad del código	10 pts	Nombres descriptivos, comentarios útiles, código sin líneas muertas, entrada validada mínimamente.
Entrega y defensa	10 pts	Repositorio GitHub actualizado con commit del proyecto. Breve explicación oral del programa y las decisiones tomadas.

Aprobado: ≥ 60 puntos. **Destacado:** ≥ 85 puntos.

Formato de salida esperado

```
=== SISTEMA DE HERRAMIENTAS DEL TALLER ===
1. Agregar herramienta
2. Listar todos los herramientas
3. Buscar herramienta por nombre
4. Mostrar herramientas activos
5. Mostrar herramientas con depósito vencida
6. Salir
Opción: 1
Nombre: Carlos
Edad: 34
Depósito: 1500
Devolución: mayo
Herramienta agregado correctamente.
```

Al terminar

Commit final con mensaje `"feat: proyecto puente taller completo"` y push al repo grupal. Prepara una breve explicación oral (2 minutos) sobre qué hace tu programa, cómo elegiste las estructuras de datos y qué función te costó más.

Evaluación del momento de intensificación y fortalecimiento 34-35

Evaluación de la instancia «Evaluación del momento de intensificación y fortalecimiento 34-35» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-intensificaciones-34-35-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-intensificaciones-34-35-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación del momento de intensificación y fortalecimiento 34-35
Momento	Intensificación y fortalecimiento — encuentros 34-35
Carácter/Objetivo	Evaluar los núcleos de las Unidades 3 y 4: procesamiento de texto, validación con <code>try / except</code> , menú en memoria, y flujo profesional del repositorio (README, issues, ramas, PR).
Destinatarios	Todo el curso
Duración teórica	240 minutos (4 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-intensificaciones-34-35.md</code> · <code>evaluacion-intensificaciones-34-35-version-a.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (240 min)

Momento	Tiempo	Qué ocurre
Apertura y organización	10 min	Presentación de la evaluación, criterios y asignación de pistas.
Bloque 1 — Texto y validación	45 min	Ejercicio de procesamiento de strings con <code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code> , y validación con <code>try / except</code> .
Bloque 2 — Mini-menú en memoria	45 min	Implementar un mini-menú con diccionario como almacenamiento en memoria.
Bloque 3 — Flujo profesional del repo	20 min	Verificación del repositorio grupal: README, issues, ramas y PRs.
Cierre y verificación individual	20 min	Registro de objetivos logrados y pendientes.
Total	240 min	

3. Acuerdo pedagógico por grupo de condición

Grupo de intensificación (contenidos mínimos irrenunciables): procesar strings con métodos de cadenas, validar entrada con `try / except`, construir un menú en memoria con diccionarios, crear o actualizar el README del repo, completar el trabajo final con commits, rama y PR. Actividad: ejercicios guiados sobre el dominio de la pizzería (pedidos, ingredientes, teléfono, precios). Recursos: VS Code, terminal, repo grupal clonado, guía impresa de métodos de cadenas y `try / except`.

Grupo de fortalecimiento: composición de métodos de cadenas para parseo complejo, validación robusta con `try / except / else / finally`, implementación de sub-menús, code review cruzado y resolución de issues con ramas y PRs. Actividad: desafíos autónomos de extensión y liderazgo de revisiones entre pares. Recursos: VS Code, terminal, repo grupal, consignas de desafío impresas.

4. Desarrollo de los encuentros del momento

Encuentro 34 (120 min): apertura conjunta (10 min) — recorrido de núcleos U3 y U4. Bloque 1 (45 min): métodos de cadenas — `split()`, `strip()`, `join()`, `replace()` sobre pedidos de pizzería (ingredientes, teléfono, nombre del comprador, precio). Bloque 2 (45 min): validación con `try / except` y menú en memoria con diccionario de funciones. Cierre conjunto (20 min): puesta en común y avances.

Encuentro 35 (120 min): apertura conjunta (10 min) — repaso de U3 y objetivos de U4. Bloque 1 (45 min): README profesional — estructura básica, creación/actualización en rama `docs/readme`, creación de issues en GitHub. Bloque 2 (45 min): trabajo final — commits en rama `feature/trabajo-final`, PR hacia `main`, code review cruzado. Cierre conjunto (20 min): verificación individual y registro de objetivos.

5. Criterios de calificación

Criterio: **Apto / No apto aún por objetivo mínimo**. No hay puntaje numérico.

Resultado	Condición
Apto	Procesa strings con <code>split / strip / join / replace</code> . Escribe f-strings con formato. Valida entrada con <code>try / except</code> capturando <code>ValueError</code> . Construye un menú en memoria con diccionarios. El repo tiene README básico. El trabajo final está commiteado con rama y PR hacia <code>main</code> .
No apto aún por objetivo mínimo	Falta al menos uno de los elementos anteriores (no procesa strings, no valida, no tiene menú, no tiene README, no completó el trabajo final con commits y PR). Se registran los objetivos pendientes y se deriva a diciembre o marzo.

6. Condiciones de resolución de la prueba

Resolución individual con computadora, VS Code y terminal. No se permite celular. El alumno escribe un único archivo `.py`. Debe tener el repo grupal clonado y el trabajo final en progreso. Se permite consultar la guía impresa de métodos de cadenas y `try / except`.

7. Regla de equivalencia entre versiones

Misma estructura, mismos objetivos y requisitos, distinto dominio (pizzería vs. veterinaria). La versión B se genera desde la versión A mediante sustitución de tokens de la tabla de dominio.

8. Mecánica de asignación de versiones

Se asignan por fila de la tabla del curso según orden de lista. Se registra la versión asignada en la planilla de calificaciones.

9. Devolución

Al cierre del encuentro 35: verificación individual y registro de objetivos logrados y pendientes. Los objetivos no alcanzados se retoman en la instancia de diciembre o marzo según el calendario académico.

Evaluación del momento 34-35 — Versión A

Dominio de esta versión: pizzería (pedidos, ingredientes, teléfono, comprador). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-34-35.md](#).

Antes de empezar

- Aprendiste a procesar texto con `split`, `strip`, `join`, `replace` y a validar entrada con `try/except`.
- También practicaste el flujo profesional del repositorio: README, issues, ramas y PR.
- Creá un archivo `pizzeria.py` en la carpeta `tp-u3/` de tu repo grupal.
- No se permite celular. Trabajo individual.
- Al terminar, hace commit y push al repo grupal.

Objetivos de la prueba

- Procesar texto de entrada con métodos de cadenas (`split`, `strip`, `join`, `replace`).
- Escribir f-strings con formato legible.
- Validar datos de entrada con `try/except` capturando `ValueError`.
- Construir un menú en memoria con diccionario como almacenamiento.
- Verificar el flujo profesional del repositorio (README, issues, ramas, PR).

Parte 1 — Procesamiento de texto de pedidos de pizzería

1A — Parsear un pedido (15 puntos conceptuales)

El texto de entrada de un **pedido** tiene este formato separado por `|`:

```
Muzzarella|Tomate|Aceitunas|Jamon
```

Escribí una función `parseOrder(orderText)` que reciba ese texto, lo divida con `split("|")`, limpie espacios con `strip()` y devuelva una lista de **ingredientes**.

Ejemplo:

```
# Llamada:
resultado = parseOrder(" Muzzarella | Tomate | Aceitunas | Jamon ")
# Devuelve:
# ["Muzzarella", "Tomate", "Aceitunas", "Jamon"]
```

1B — Mostrar detalle del pedido (15 puntos conceptuales)

Escribí una función `mostrar_pedido(comprador, ingredientes, orderPrice)` que reciba el nombre del **comprador**, la lista de **ingredientes** y el **precio** (`orderPrice`), y muestre usando f-strings:

```
Pedido de Ana:
Ingredientes: Muzzarella, Tomate, Aceitunas, Jamon
Precio: $1200.00
Teléfono de contacto: 1145678901
```

Usá `join(", ")` para unir los ingredientes.

1C — Validar teléfono (15 puntos conceptuales)

Escribí una función `validPhone(telefono)` que verifique que el string tenga exactamente 10 caracteres y todos sean dígitos (usá `len()` y `isdigit()`). Si no cumple, debe devolver `False`. Integra esta validación en el programa.

Parte 2 — Menú en memoria con pedidos

Implementá un mini-menú que permita:

```
=== PIZZERÍA ===
1. Cargar pedido
2. Ver todos los pedidos
3. Buscar pedido por nombre del comprador
4. Salir
```

Los **pedidos** se guardan en un diccionario donde la clave es el nombre del **comprador** (`customerName`) y el valor es otro diccionario con los datos del pedido: ingredientes, precio y teléfono.

Usá `try/except` al cargar el **precio** (`orderPrice`) para que, si el usuario ingresa un texto no numérico, el programa muestre “Precio inválido. Ingrese un número.” y pida el dato de nuevo.

Función validada con `try/except`

```
def read_float(mensaje):
    while True:
        try:
            return float(input(mensaje))
        except ValueError:
            print("Precio inválido. Ingrese un número.")
```

Parte 3 — Flujo profesional del repositorio (verificación)

Además del código en `tp-u3/pizzeria.py`, verificá que en el repositorio grupal:

1. El README.md tenga al menos título, descripción y cómo ejecutar el programa.
2. Exista al menos un **issue** abierto con una tarea (por ejemplo, “Agregar opción de modificar pedido”).
3. El **trabajo final** esté en una rama `feature/trabajo-final` con un **pull request** abierto hacia `main`.
4. Se haya hecho al menos un code review (comentario en el PR).

Al terminar

Commit con mensaje `"feat: evaluacion 34-35 pizzeria"` y push al repo grupal. Avisá al docente para que verifique tanto el código como el flujo profesional del repositorio.

Evaluación del momento 34-35 — Versión A

Dominio de esta versión: veterinaria (pedidos, vacunas, teléfono, mascota). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-34-35.md](#).

Antes de empezar

- Aprendiste a procesar texto con `split`, `strip`, `join`, `replace` y a validar entrada con `try/except`.
- También practicaste el flujo profesional del repositorio: README, issues, ramas y PR.
- Creá un archivo `pizzeria.py` en la carpeta `tp-u3/` de tu repo grupal.
- No se permite celular. Trabajo individual.
- Al terminar, hace commit y push al repo grupal.

Objetivos de la prueba

- Procesar texto de entrada con métodos de cadenas (`split`, `strip`, `join`, `replace`).
- Escribir f-strings con formato legible.
- Validar datos de entrada con `try/except` capturando `ValueError`.
- Construir un menú en memoria con diccionario como almacenamiento.
- Verificar el flujo profesional del repositorio (README, issues, ramas, PR).

Parte 1 — Procesamiento de texto de pedidos de veterinaria

1A — Parsear un turno (15 puntos conceptuales)

El texto de entrada de un **turno** tiene este formato separado por `|`:

```
Muzzarella|Tomate|Aceitunas|Jamon
```

Escribí una función `parseAppointment(appointmentText)` que reciba ese texto, lo divida con `split("|")`, limpie espacios con `strip()` y devuelva una lista de **vacunas**.

Ejemplo:

```
# Llamada:
resultado = parseAppointment(" Muzzarella | Tomate | Aceitunas | Jamon ")
# Devuelve:
# ["Muzzarella", "Tomate", "Aceitunas", "Jamon"]
```

1B — Mostrar detalle del turno (15 puntos conceptuales)

Escribí una función `mostrar_pedido(mascota, vacunas, consultPrice)` que reciba el nombre del **mascota**, la lista de **vacunas** y el **tarifa** (`consultPrice`), y muestre usando f-strings:

```
Turno de Ana:
Vacunas: Muzzarella, Tomate, Aceitunas, Jamon
Tarifa: $1200.00
Teléfono de contacto: 1145678901
```

Usá `join(", ")` para unir los vacunas.

1C — Validar teléfono (15 puntos conceptuales)

Escribí una función `validName(historial)` que verifique que el string tenga exactamente 10 caracteres y todos sean dígitos (usá `len()` y `isdigit()`). Si no cumple, debe devolver `False`. Integrá esta validación en el programa.

Parte 2 — Menú en memoria con pedidos

Implementá un mini-menú que permita:

```
=== VETERINARIA ===
1. Cargar turno
2. Ver todos los pedidos
3. Buscar turno por nombre del mascota
4. Salir
```

Los **pedidos** se guardan en un diccionario donde la clave es el nombre del **mascota** (`petName`) y el valor es otro diccionario con los datos del turno: vacunas, tarifa y teléfono.

Usá `try/except` al cargar el **tarifa** (`consultPrice`) para que, si el usuario ingresa un texto no numérico, el programa muestre “Tarifa inválido. Ingrese un número.” y pida el dato de nuevo.

Función validada con `try/except`

```
def read_float(mensaje):
    while True:
        try:
            return float(input(mensaje))
        except ValueError:
            print("Tarifa inválido. Ingrese un número.")
```

Parte 3 — Flujo profesional del repositorio (verificación)

Además del código en `tp-u3/pizzeria.py`, verificá que en el repositorio grupal:

1. El README.md tenga al menos título, descripción y cómo ejecutar el programa.
2. Exista al menos un **issue** abierto con una tarea (por ejemplo, “Agregar opción de modificar turno”).
3. El **trabajo final** esté en una rama `feature/trabajo-final` con un **pull request** abierto hacia `main`.
4. Se haya hecho al menos un code review (comentario en el PR).

Al terminar

Commit con mensaje `"feat: evaluacion 34-35 pizzeria"` y push al repo grupal. Avisá al docente para que verifique tanto el código como el flujo profesional del repositorio.

Evaluación del momento de intensificación de diciembre — Camino mínimo completo

Evaluación de la instancia «Intensificación de diciembre — Camino mínimo completo» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-intensificaciones-diciembre-version-a` y sus versiones equivalentes generadas.

1. Identificación

Campo	Detalle
Instancia	Intensificación de diciembre — Camino mínimo completo
Momento	Intensificación de diciembre (fuera de la estructura anual)
Carácter/Objetivo	Evaluar el camino mínimo completo del curso mediante un programa integrador de consola con defensa oral. Dominio: verdulería (productos, kilos, precio, factura, stock).
Destinatarios	Estudiantes que no alcanzaron los objetivos mínimos del curso durante el ciclo lectivo
Duración teórica	240 minutos (4 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-intensificaciones-diciembre.md</code> · <code>evaluacion-intensificaciones-diciembre-version-a.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (240 min)

Momento	Tiempo	Qué ocurre
Apertura y organización	10 min	Presentación de la instancia: camino mínimo completo, criterio Apto/No apto.
Bloque 1 — Entrada, tipos y condicionales	45 min	Ejercicios 1 a 3: programa base con variables, condicionales y menú simple con <code>while</code> .
Bloque 2 — Listas, diccionarios y bucles	45 min	Ejercicios 4 a 6: lista de productos como diccionarios, recorrido con <code>for</code> , búsqueda por nombre.
Bloque 3 — Funciones, validación y menú	45 min	Ejercicios 7 a 10: refactorización con funciones, bloque principal, <code>try / except</code> , menú completo.
Bloque 4 — Entrega y defensa	45 min	Ejercicios 11 y 12: commit final, README básico, defensa individual.
Total	240 min	

3. Acuerdo pedagógico

Dimensión	Grupo de intensificación (recuperación pedagógica) — único grupo
Contenidos	Camino mínimo completo: 1) entrada/salida con <code>print()</code> / <code>input()</code> ; 2) variables y tipos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>); 3) condicionales (<code>if</code> / <code>elif</code> / <code>else</code>); 4) bucles (<code>for</code> , <code>while</code>); 5) listas y diccionarios; 6) funciones con <code>def</code> , parámetros y <code>return</code> ; 7) bloque principal (<code>if __name__ == "__main__"</code>); 8) métodos de cadenas (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>), f-strings; 9) validación con <code>try</code> / <code>except</code> ; 10) menú en memoria; 11) entregas por GitHub (carpetas <code>tp-u1/</code> a <code>trabajo-final/</code>); 12) defensa oral. Dominio: verdulería (productos, kilos, precio, factura, stock).
Actividad / metodología	Recorrido acelerado por los contenidos mínimos del año. Cada estudiante reconstruye o completa un programa integrador de consola sobre verdulería en dos encuentros.
Recursos	VS Code, terminal, repo grupal clonado, guía paso a paso del camino mínimo (impresa), lista de verificación de objetivos por alumno, plantilla README.

4. Desarrollo de los encuentros del momento

Encuentro 1 (120 min): apertura (10 min) — explicación de la modalidad, entrega de lista de verificación individual. Bloque 1 (45 min): programa base (`verduleria.py`) con variables, `input()` con conversión, condicionales de stock bajo y descuento, menú simple con `while`. Bloque 2 (45 min): lista de productos como diccionarios, agregar con `append()`, mostrar con `for`, buscar por nombre. Cierre (20 min): verificación individual, commit con mensaje `"feat: verduleria base y listado"`.

Encuentro 2 (120 min): apertura (10 min) — repaso y objetivos del día. Bloque 1 (45 min): refactorización con funciones (`agregar_producto`, `listar_productos`, `buscar_producto`, `vender_producto`), bloque principal `if __name__`, validación con `try` / `except`. Bloque 2 (45 min): menú completo (opciones 1 a 6: agregar, listar, buscar, vender, factura, salir), commit final, README básico. Cierre con defensa (20 min): recorrido por los puestos con lista de verificación.

5. Criterios de calificación

Criterio: **Apto / No apto aún por objetivo mínimo**. No hay puntaje numérico.

Resultado	Condición
Apto	El programa compila y ejecuta. Usa <code>input()</code> / <code>print()</code> , condicionales, bucles, listas, diccionarios, funciones, bloque principal, <code>try</code> / <code>except</code> , f-strings y menú en memoria. El repo tiene commit y README. El estudiante explica su código durante la defensa.
No apto aún por objetivo mínimo	Falta al menos uno de los elementos anteriores. Se especifica qué objetivo(s) no se alcanzó y se sugiere la instancia de marzo como siguiente oportunidad con el mismo estándar.

6. Condiciones de resolución de la prueba

Resolución individual con computadora, VS Code y terminal. No se permite celular. El alumno escribe un único archivo `.py`. Debe tener el repo grupal clonado con su cuenta de GitHub activa. Se permite consultar la guía paso a paso del camino mínimo.

7. Regla de equivalencia entre versiones

Misma estructura, mismos objetivos y requisitos, distinto dominio (verdulería vs. videoteca). La versión B se genera desde la versión A mediante sustitución de tokens de la tabla de dominio.

8. Mecánica de asignación de versiones

Se asignan por fila de la tabla del curso según orden de lista. Se registra la versión asignada en la planilla de calificaciones.

9. Devolución

Al finalizar la defensa: verificación individual con la lista de objetivos, registro de Apto o No apto con detalle de pendientes. Si es No apto, se informa la instancia de marzo como siguiente oportunidad con el mismo estándar, y se entrega la lista de objetivos pendientes para que el estudiante se prepare.

Evaluación de diciembre — Versión A

Dominio de esta versión: verdulería (productos, kilos, precio, factura, stock). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-diciembre.md](#).

Antes de empezar

- Esta evaluación verifica el **camino mínimo completo del curso**. Cada objetivo que logres hoy es un objetivo aprobado.
- Creá un archivo `verduleria.py` en la carpeta `trabajo-final/` de tu repo grupal.
- Escribí el programa en un solo archivo `.py`. Funciones arriba, `if __name__ == "__main__":` al final.
- No se permite celular. Trabajo individual.
- Al terminar, commit y push al repo grupal, y preparate para la defensa oral.

Objetivos de la prueba

- Usar `print()` / `input()` con conversión de tipos (`int()`, `float()`).
- Aplicar condicionales `if` / `elif` / `else`.
- Usar bucles `while` (menú) y `for` (recorrer colecciones).
- Almacenar datos en listas de diccionarios.
- Definir funciones con `def`, parámetros y `return`.
- Validar entrada con `try` / `except`.
- Usar f-strings y métodos de cadenas básicos.
- Entregar en GitHub con commit y README.

Consigna — Programa integrador de verdulería

Escribí un programa que gestione los **productos** de una **verdulería**. Cada **producto** es un diccionario con:

- `"nombre"`: string
- `"precio"` (`productPrice`): float (precio por kilo)
- `"kilos"` (`stockWeight`): float (kilos disponibles en stock)

Menú completo (con `while`)

```
=== VERDULERÍA ===
1. Agregar producto
2. Listar productos
3. Buscar producto por nombre
4. Vender producto
5. Factura
6. Salir
```

Funciones requeridas

1. `agregar_producto(productList)` — pide nombre, precio por kilo y kilos disponibles. Valida que precio y kilos sean números con `try` / `except`. Agrega el producto como diccionario a la lista.
2. `listar_productos(productList)` — recorre la lista con `for` y muestra cada producto.

3. `buscar_producto(productList, productName)` — busca por nombre exacto, devuelve el diccionario o `None` si no existe.
4. `vender_producto(productList, ventas) (sellProduct)` — pide nombre del producto y kilos a vender. Verifica que haya stock suficiente. Si el producto tiene menos de 1 kilo, muestra “Stock bajo”. Si se vende, descuenta los kilos y agrega la venta a una lista de ventas.
5. `mostrar_factura(ventas)` — recorre la lista de ventas y muestra cada una con el total. Usá f-strings con formato de dos decimales.

Función de validación con `try/except`

```
def read_float(mensaje):  
    while True:  
        try:  
            return float(input(mensaje))  
        except ValueError:  
            print("Dato inválido. Ingrese un número.")
```

Formato de salida esperado

```
=== VERDULERÍA ===  
1. Agregar producto  
2. Listar productos  
3. Buscar producto por nombre  
4. Vender producto  
5. Factura  
6. Salir  
Opción: 1  
Nombre del producto: Tomate  
Precio por kilo: 250.50  
Kilos disponibles: 15  
Producto agregado correctamente.  
  
=== VERDULERÍA ===  
1. Agregar producto  
2. Listar productos  
3. Buscar producto por nombre  
4. Vender producto  
5. Factura  
6. Salir  
Opción: 4  
Nombre del producto: Tomate  
Kilos a vender: 2  
Venta realizada: 2 kilos de Tomate por $501.00
```

Defensa oral (se evalúa durante el cierre)

Preparate para explicar brevemente (2-3 minutos): - ¿Qué hace tu programa? - ¿Cómo validás los datos de entrada? - Elegí una función que te haya costado — ¿cómo la resolviste?

Al terminar

1. Commit final con mensaje `"feat: verduleria completa camino mínimo"`.
2. Push al repo grupal en la carpeta `trabajo-final/`.
3. Verificá que el README.md tenga al menos título, descripción y cómo ejecutar.
4. Avisá al docente. Te va a hacer la verificación individual y la defensa oral.

Evaluación de diciembre — Versión A

Dominio de esta versión: videoteca (películas, copias, tarifa, alquiler, ejemplares). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-diciembre.md](#).

Antes de empezar

- Esta evaluación verifica el **camino mínimo completo del curso**. Cada objetivo que logres hoy es un objetivo aprobado.
- Creá un archivo `verduleria.py` en la carpeta `trabajo-final/` de tu repo grupal.
- Escribí el programa en un solo archivo `.py`. Funciones arriba, `if __name__ == "__main__":` al final.
- No se permite celular. Trabajo individual.
- Al terminar, commit y push al repo grupal, y preparate para la defensa oral.

Objetivos de la prueba

- Usar `print()` / `input()` con conversión de tipos (`int()`, `float()`).
- Aplicar condicionales `if` / `elif` / `else`.
- Usar bucles `while` (menú) y `for` (recorrer colecciones).
- Almacenar datos en listas de diccionarios.
- Definir funciones con `def`, parámetros y `return`.
- Validar entrada con `try` / `except`.
- Usar f-strings y métodos de cadenas básicos.
- Entregar en GitHub con commit y README.

Consigna — Programa integrador de videoteca

Escribí un programa que gestione los **películas** de una **videoteca**. Cada **película** es un diccionario con:

- `"nombre"`: string
- `"tarifa"` (`moviePrice`): float (tarifa por kilo)
- `"copias"` (`stockCopies`): float (copias disponibles en ejemplares)

Menú completo (con `while`)

```
=== VIDEOTECA ===
1. Agregar película
2. Listar películas
3. Buscar película por nombre
4. Vender película
5. Alquiler
6. Salir
```

Funciones requeridas

1. `agregar_producto(movieList)` — pide nombre, tarifa por kilo y copias disponibles. Valida que tarifa y copias sean números con `try` / `except`. Agrega el película como diccionario a la lista.
2. `listar_productos(movieList)` — recorre la lista con `for` y muestra cada película.

3. `buscar_producto(movieList, movieTitle)` — busca por nombre exacto, devuelve el diccionario o `None` si no existe.
4. `vender_producto(movieList, ventas) (rentMovie)` — pide nombre del película y copias a vender. Verifica que haya ejemplares suficiente. Si el película tiene menos de 1 kilo, muestra “Ejemplares bajo”. Si se vende, descuenta los copias y agrega la préstamo a una lista de ventas.
5. `mostrar_factura(ventas)` — recorre la lista de ventas y muestra cada una con el total. Usá f-strings con formato de dos decimales.

Función de validación con `try/except`

```
def read_float(mensaje):  
    while True:  
        try:  
            return float(input(mensaje))  
        except ValueError:  
            print("Dato inválido. Ingrese un número.")
```

Formato de salida esperado

```
=== VIDEOTECA ===  
1. Agregar película  
2. Listar películas  
3. Buscar película por nombre  
4. Vender película  
5. Alquiler  
6. Salir  
Opción: 1  
Nombre del película: Tomate  
Tarifa por kilo: 250.50  
Copias disponibles: 15  
Película agregado correctamente.  
  
=== VIDEOTECA ===  
1. Agregar película  
2. Listar películas  
3. Buscar película por nombre  
4. Vender película  
5. Alquiler  
6. Salir  
Opción: 4  
Nombre del película: Tomate  
Copias a vender: 2  
Préstamo realizada: 2 copias de Tomate por $501.00
```

Defensa oral (se evalúa durante el cierre)

Preparate para explicar brevemente (2-3 minutos): - ¿Qué hace tu programa? - ¿Cómo validás los datos de entrada? - Elegí una función que te haya costado — ¿cómo la resolviste?

Al terminar

1. Commit final con mensaje `"feat: verduleria completa camino mínimo"`.
2. Push al repo grupal en la carpeta `trabajo-final/`.
3. Verificá que el README.md tenga al menos título, descripción y cómo ejecutar.
4. Avisá al docente. Te va a hacer la verificación individual y la defensa oral.

Evaluación del momento de intensificación de marzo — Camino mínimo completo

Evaluación de la instancia «Intensificación de marzo — Camino mínimo completo» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-intensificaciones-marzo-version-a` y sus versiones equivalentes generadas.

1. Identificación

Campo	Detalle
Instancia	Intensificación de marzo — Camino mínimo completo
Momento	Intensificación de marzo (fuera de la estructura anual, previa al nuevo ciclo)
Carácter/Objetivo	Evaluar el camino mínimo completo del curso (mismo estándar que diciembre). Cambia el tiempo de preparación del estudiante (diciembre → marzo). Programa integrador de consola con defensa oral. Dominio: verdulería (productos, kilos, precio, factura, stock).
Destinatarios	Estudiantes que no alcanzaron los objetivos mínimos en la instancia de diciembre
Duración teórica	240 minutos (4 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-intensificaciones-marzo.md</code> · <code>evaluacion-intensificaciones-marzo-version-a.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (240 min)

Momento	Tiempo	Qué ocurre
Apertura y organización	10 min	Presentación de la instancia, mismo estándar que diciembre. Se entrega la lista de verificación individual con los pendientes de diciembre.
Bloque 1 — Verificación de entrada, tipos, condicionales	45 min	Verificación 1 a 3: el estudiante muestra su programa base, condicionales y menú simple. Se asiste en lo que falte.
Bloque 2 — Verificación de listas, diccionarios y bucles	45 min	Verificación 4 a 6: lista de productos como diccionarios, mostrar con <code>for</code> , buscar por nombre.
Bloque 3 — Verificación de funciones, validación y menú completo	45 min	Verificación 7 a 10: funciones, bloque principal, <code>try / except</code> , menú completo.
Bloque 4 — Entrega y defensa	45 min	Verificación 11 y 12: commit final, README, defensa individual.
Total	240 min	

3. Acuerdo pedagógico

Dimensión	Grupo de intensificación (recuperación pedagógica) — único grupo
Contenidos	Camino mínimo completo del curso (idéntico al de diciembre): 1) entrada/salida con <code>print()</code> / <code>input()</code> ; 2) variables y tipos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>); 3) condicionales (<code>if / elif / else</code>); 4) bucles (<code>for</code> , <code>while</code>); 5) listas y diccionarios; 6) funciones con <code>def</code> , parámetros y <code>return</code> ; 7) bloque principal (<code>if __name__ == "__main__"</code>); 8) métodos de cadenas (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>), f-strings; 9) validación con <code>try / except</code> ; 10) menú en memoria; 11) entregas por GitHub (carpetas <code>tp-u1/</code> a <code>trabajo-final/</code>); 12) defensa oral. Dominio: verdulería (productos, kilos, precio, factura, stock).
Actividad / metodología	El estudiante tuvo más tiempo para prepararse (diciembre a marzo). La instancia presencial se organiza como taller de verificación: el docente revisa el programa que el estudiante trae preparado y asiste en los puntos que todavía presenten dificultad.
Recursos	VS Code, terminal, repo grupal clonado, guía paso a paso del camino mínimo (impresa desde diciembre), lista de verificación de objetivos por alumno con el registro de diciembre, plantilla README.

4. Desarrollo de los encuentros del momento

Encuentro 1 (120 min): apertura (10 min) — bienvenida, explicación de que el estándar es el mismo que en diciembre, entrega de lista de verificación individual actualizada. Bloque 1 (45 min): verificación 1 a 3 — el estudiante muestra su programa base funcionando, condicionales con stock bajo y descuento, menú simple con `while`. Bloque 2 (45 min): verificación 4 a 6 — lista de productos como diccionarios, mostrar con `for`, buscar por nombre. Cierre (20 min): registro de objetivos logrados, commit si hubo modificaciones.

Encuentro 2 (120 min): apertura (10 min) — repaso de objetivos de funciones, validación y entrega. Bloque 1 (45 min): verificación 7 a 9 — funciones (`agregar_producto` , `listar_productos` , `buscar_producto` , `vender_producto`), bloque principal, `try / except` capturando `ValueError` . Bloque 2 (45 min): verificación 10 a 12 — menú completo (opciones 1 a 6), commit final con mensaje `"feat: verduleria completa camino mínimo"` , README básico. Cierre con defensa (20 min): recorrido por los puestos con lista de verificación.

5. Criterios de calificación

Criterio: **Apto / No apto aún por objetivo mínimo** (mismo estándar que diciembre). No baja el estándar; cambia el tiempo de preparación del estudiante.

Resultado	Condición
Apto	El programa compila y ejecuta. Usa <code>input()</code> / <code>print()</code> , condicionales, bucles, listas, diccionarios, funciones, bloque principal, <code>try / except</code> , f-strings y menú en memoria. El repo tiene commit y README. El estudiante explica su código durante la defensa.
No apto aún por objetivo mínimo	Falta al menos uno de los elementos anteriores. Se especifica qué objetivo(s) no se alcanzó y se informa al estudiante junto con la notificación a dirección.

6. Condiciones de resolución de la prueba

Resolución individual con computadora, VS Code y terminal. No se permite celular. El alumno trae su programa preparado desde diciembre y lo muestra en vivo. El docente verifica cada objetivo. Si falta algo, se asiste en la resolución durante el encuentro. Se permite consultar la guía paso a paso del camino mínimo.

7. Regla de equivalencia entre versiones

Misma estructura, mismos objetivos y requisitos, distinto dominio (verdulería vs. videoteca). La versión B se genera desde la versión A mediante sustitución de tokens de la tabla de dominio.

8. Mecánica de asignación de versiones

Se asignan por fila de la tabla del curso según orden de lista. Se registra la versión asignada en la planilla de calificaciones.

9. Devolución

Al finalizar la defensa: verificación individual con la lista de objetivos, registro de Apto o No apto con detalle de pendientes. Si es No apto, se entrega el detalle de objetivos pendientes y se informa al estudiante junto con la comunicación formal del resultado a dirección. No hay instancia posterior: esta es la última oportunidad del ciclo.

Evaluación de marzo — Versión A

Dominio de esta versión: verdulería (productos, kilos, precio, factura, stock). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-marzo.md](#).

Antes de empezar

- Esta evaluación verifica el **camino mínimo completo del curso** con el **mismo estándar que diciembre**. La única diferencia es el tiempo de preparación: tuviste de diciembre a marzo para practicar.
- Creá un archivo `verduleria.py` en la carpeta `trabajo-final/` de tu repo grupal (o traelo preparado de tu casa).
- Escribí el programa en un solo archivo `.py`. Funciones arriba, `if __name__ == "__main__":` al final.
- No se permite celular. Trabajo individual.
- Al terminar, commit y push al repo grupal, y preparate para la defensa oral.

Objetivos de la prueba

- Usar `print()` / `input()` con conversión de tipos (`int()`, `float()`).
- Aplicar condicionales `if` / `elif` / `else`.
- Usar bucles `while` (menú) y `for` (recorrer colecciones).
- Almacenar datos en listas de diccionarios.
- Definir funciones con `def`, parámetros y `return`.
- Validar entrada con `try` / `except`.
- Usar f-strings y métodos de cadenas básicos.
- Entregar en GitHub con commit y README.

Consigna — Programa integrador de verdulería (mismo estándar que diciembre)

Escribí un programa que gestione los **productos** de una **verdulería**. Cada **producto** es un diccionario con:

- `"nombre"`: string
- `"precio"` (`productPrice`): float (precio por kilo)
- `"kilos"` (`stockWeight`): float (kilos disponibles en stock)

Menú completo (con `while`)

```
=== VERDULERÍA ===
1. Agregar producto
2. Listar productos
3. Buscar producto por nombre
4. Vender producto
5. Factura
6. Salir
```

Funciones requeridas

1. `agregar_producto(productList)` — pide nombre, precio por kilo y kilos disponibles. Valida que precio y kilos sean números con `try` / `except`. Agrega el producto como diccionario a la lista.

2. `listar_productos(productList)` — recorre la lista con `for` y muestra cada producto.
3. `buscar_producto(productList, productName)` — busca por nombre exacto, devuelve el diccionario o `None` si no existe.
4. `vender_producto(productList, ventas)` (`sellProduct`) — pide nombre del producto y kilos a vender. Verifica que haya stock suficiente. Si el producto tiene menos de 1 kilo, muestra “Stock bajo”. Si se vende, descuenta los kilos y agrega la venta a una lista de ventas.
5. `mostrar_factura(ventas)` — recorre la lista de ventas y muestra cada una con el total. Usá f-strings con formato de dos decimales.

Función de validación con `try / except`

```
def read_float(mensaje):  
    while True:  
        try:  
            return float(input(mensaje))  
        except ValueError:  
            print("Dato inválido. Ingrese un número.")
```

Formato de salida esperado

```
=== VERDULERÍA ===  
1. Agregar producto  
2. Listar productos  
3. Buscar producto por nombre  
4. Vender producto  
5. Factura  
6. Salir  
Opción: 1  
Nombre del producto: Tomate  
Precio por kilo: 250.50  
Kilos disponibles: 15  
Producto agregado correctamente.  
  
=== VERDULERÍA ===  
1. Agregar producto  
2. Listar productos  
3. Buscar producto por nombre  
4. Vender producto  
5. Factura  
6. Salir  
Opción: 4  
Nombre del producto: Tomate  
Kilos a vender: 2  
Venta realizada: 2 kilos de Tomate por $501.00
```

Nota sobre el estándar

Este programa es idéntico en requisitos al de la instancia de diciembre. **No baja el estándar:** la única diferencia es que tuviste tiempo adicional para prepararte. Si aprobabas en diciembre con este mismo programa, aprobás ahora; si no, tenés esta nueva oportunidad con el mismo nivel de exigencia.

Defensa oral (se evalúa durante el cierre)

Preparate para explicar brevemente (2-3 minutos): - ¿Qué hace tu programa? - ¿Cómo validás los datos de entrada? - ¿Cuál es la diferencia entre una lista y un diccionario en tu programa? - Elegí una función que te haya costado — ¿cómo la resolviste?

Al terminar

1. Commit final con mensaje `"feat: verduleria completa camino mínimo marzo"`.

2. Push al repo grupal en la carpeta `trabajo-final/`.
3. Verificá que el README.md tenga al menos título, descripción y cómo ejecutar.
4. Avisá al docente. Te va a hacer la verificación individual y la defensa oral.

Evaluación de marzo — Versión A

Dominio de esta versión: videoteca (películas, copias, tarifa, alquiler, ejemplares). Duración: 90 minutos. Puntaje total: apto/no apto por objetivo mínimo. Resolución individual, con computadora, sin celular. Las condiciones completas están en [evaluacion-intensificaciones-marzo.md](#).

Antes de empezar

- Esta evaluación verifica el **camino mínimo completo del curso** con el **mismo estándar que diciembre**. La única diferencia es el tiempo de preparación: tuviste de diciembre a marzo para practicar.
- Creá un archivo `verduleria.py` en la carpeta `trabajo-final/` de tu repo grupal (o traelo preparado de tu casa).
- Escribí el programa en un solo archivo `.py`. Funciones arriba, `if __name__ == "__main__":` al final.
- No se permite celular. Trabajo individual.
- Al terminar, commit y push al repo grupal, y preparate para la defensa oral.

Objetivos de la prueba

- Usar `print()` / `input()` con conversión de tipos (`int()`, `float()`).
- Aplicar condicionales `if/elif/else`.
- Usar bucles `while` (menú) y `for` (recorrer colecciones).
- Almacenar datos en listas de diccionarios.
- Definir funciones con `def`, parámetros y `return`.
- Validar entrada con `try/except`.
- Usar f-strings y métodos de cadenas básicos.
- Entregar en GitHub con commit y README.

Consigna — Programa integrador de videoteca (mismo estándar que diciembre)

Escribí un programa que gestione los **películas** de una **videoteca**. Cada **película** es un diccionario con:

- `"nombre"`: string
- `"tarifa"` (`moviePrice`): float (tarifa por kilo)
- `"copias"` (`stockCopies`): float (copias disponibles en ejemplares)

Menú completo (con `while`)

```
=== VIDEOTECA ===
1. Agregar película
2. Listar películas
3. Buscar película por nombre
4. Vender película
5. Alquiler
6. Salir
```

Funciones requeridas

1. `agregar_producto(movieList)` — pide nombre, tarifa por kilo y copias disponibles. Valida que tarifa y copias sean números con `try/except`. Agrega el película como diccionario a la lista.

2. `listar_productos(movieList)` — recorre la lista con `for` y muestra cada película.
3. `buscar_producto(movieList, movieTitle)` — busca por nombre exacto, devuelve el diccionario o `None` si no existe.
4. `vender_producto(movieList, ventas) (rentMovie)` — pide nombre de la película y copias a vender. Verifica que haya ejemplares suficientes. Si la película tiene menos de 1 kilo, muestra “Ejemplares bajo”. Si se vende, descuenta las copias y agrega la préstamo a una lista de ventas.
5. `mostrar_factura(ventas)` — recorre la lista de ventas y muestra cada una con el total. Usá f-strings con formato de dos decimales.

Función de validación con `try/except`

```
def read_float(mensaje):
    while True:
        try:
            return float(input(mensaje))
        except ValueError:
            print("Dato inválido. Ingrese un número.")
```

Formato de salida esperado

```
=== VIDEOTECA ===
1. Agregar película
2. Listar películas
3. Buscar película por nombre
4. Vender película
5. Alquiler
6. Salir
Opción: 1
Nombre del película: Tomate
Tarifa por kilo: 250.50
Copias disponibles: 15
Película agregado correctamente.

=== VIDEOTECA ===
1. Agregar película
2. Listar películas
3. Buscar película por nombre
4. Vender película
5. Alquiler
6. Salir
Opción: 4
Nombre del película: Tomate
Copias a vender: 2
Préstamo realizada: 2 copias de Tomate por $501.00
```

Nota sobre el estándar

Este programa es idéntico en requisitos al de la instancia de diciembre. **No baja el estándar:** la única diferencia es que tuviste tiempo adicional para prepararte. Si aprobabas en diciembre con este mismo programa, aprobás ahora; si no, tenés esta nueva oportunidad con el mismo nivel de exigencia.

Defensa oral (se evalúa durante el cierre)

Preparate para explicar brevemente (2-3 minutos): - ¿Qué hace tu programa? - ¿Cómo validás los datos de entrada? - ¿Cuál es la diferencia entre una lista y un diccionario en tu programa? - Elegí una función que te haya costado — ¿cómo la resolviste?

Al terminar

1. Commit final con mensaje `"feat: verduleria completa camino mínimo marzo"`.

2. Push al repo grupal en la carpeta `trabajo-final/`.
3. Verificá que el README.md tenga al menos título, descripción y cómo ejecutar.
4. Avisá al docente. Te va a hacer la verificación individual y la defensa oral.

Intensificación y fortalecimiento de saberes previos — Encuentros 2 y 3

Metadatos

Campo	Valor
Momento	Intensificación y fortalecimiento de saberes previos
Encuentros	2 y 3
Duración	2 encuentros × 120 min (240 min total)
Destinatarios	Totalidad del curso, con pistas diferenciadas por condición
Requisitos	Diagnóstico del Encuentro 1 resuelto; resultados individuales de nivelación
Lugar	Aula de informática con VS Code y terminal

Acuerdo pedagógico

Dimensión	Grupo de intensificación (recuperación pedagógica)	Grupo de fortalecimiento (profundización)
Contenidos	Manejo de PC: archivos/carpetas, navegación en el sistema. Terminal: comandos básicos (<code>cd</code> , <code>dir</code> / <code>ls</code> , <code>mkdir</code>). Lógica de programación general: secuencia, decisión, repetición (diagramas de flujo, sin Python aún).	Desafíos de lógica sin código: problemas de secuencia, decisión y repetición resueltos en papel y simulados en terminal. Primeros pasos documentados: bitácora personal de comandos y algoritmos.
Actividad / metodología	Ejercicios guiados de terminal: navegar por carpetas, crear estructura de directorios del curso. Traducir algoritmos cotidianos a diagramas de flujo. Corrección paso a paso.	Desafíos de lógica (torre de Hanói, búsqueda en laberinto) resueltos como pseudocódigo. Bitácora de comandos de terminal con ejemplos propios.
Recursos	VS Code, terminal del sistema, guía impresa de comandos básicos, plantilla de diagrama de flujo.	VS Code, terminal del sistema, consignas de desafío impresas, plantilla de bitácora personal.

Desarrollo del Encuentro 2

Apertura conjunta (10 min)

Plenaria: el docente presenta los resultados del diagnóstico del Encuentro 1 y explica la organización en dos pistas (intensificación / fortalecimiento). Cada estudiante recibe su asignación. Se recuerda que **no se permite celular** y que todo el trabajo se hace en la PC.

Pista intensificación (45 min + 45 min)

Bloque 1 — Terminal y archivos (45 min) 1. **(15 min)** Conceptos: sistema de archivos, ruta absoluta vs relativa, carpeta vs archivo. Comandos `pwd` , `cd` , `ls` (Linux) o `dir` (Windows). 2. **(15 min)** Práctica guiada: navegar por el sistema, crear la estructura de carpetas del curso (`LAP/` , `LAP/unidad-1/` , etc.) desde la terminal. 3. **(15 min)** Ejercicio: dado un enunciado escrito, crear la estructura de carpetas correspondiente usando solo comandos.

Bloque 2 — Lógica de programación (45 min) 1. **(15 min)** Conceptos de secuencia, decisión y repetición con ejemplos cotidianos (receta de cocina, semáforo, contar boletos). 2. **(15 min)** Diagrama de flujo: símbolos básicos (inicio/fin, proceso, decisión, bucle). Traducir “preparar un mate” a diagrama. 3. **(15 min)** Ejercicio individual: dibujar el diagrama de flujo de “decidir si llevar paraguas según el clima”.

Pista fortalecimiento (45 min + 45 min)

Bloque 1 — Desafíos de terminal (45 min) 1. **(15 min)** Crear estructura anidada con un solo comando (`mkdir -p` o equivalente). Renombrar y mover archivos desde terminal. 2. **(15 min)** Desafío: replicar el árbol de directorios de un proyecto Python real (carpetas `src/`, `tests/`, `docs/`) usando una sola línea de comandos encadenados. 3. **(15 min)** Bitácora: registrar los comandos nuevos aprendidos con ejemplos de uso real.

Bloque 2 — Desafíos de lógica (45 min) 1. **(15 min)** Problema de secuencia: describir paso a paso cómo ordenar tres números sin usar código. 2. **(15 min)** Problema de decisión compuesta: dado un horario y un día, decidir si el kiosco escolar está abierto (combinación de condiciones). 3. **(15 min)** Problema de repetición: simular en terminal la cuenta regresiva con `echo` dentro de un bucle conceptual.

Cierre conjunto (20 min)

Puesta en común: un voluntario de cada pista comparte su diagrama / bitácora. El docente destaca que la lógica de secuencia → decisión → repetición es la base de TODO programa, sin importar el lenguaje. Anticipa que en el próximo encuentro conocerán la plataforma de entregas.

Desarrollo del Encuentro 3

Apertura conjunta (10 min)

Repaso rápido: ¿qué es una ruta? ¿qué significa repetir? El docente escribe un diagrama de flujo en el pizarrón y lo recorre colectivamente. Anuncia que hoy se suma la plataforma de entregas del curso.

Pista intensificación (45 min + 45 min)

Bloque 1 — Plataforma de entregas (45 min) 1. **(10 min)** El docente proyecta la URL del repositorio grupal de GitHub (organización del curso). Explica la diferencia entre navegador y cuenta propia. 2. **(15 min)** Cada estudiante abre el navegador, ingresa la URL, inicia sesión en GitHub con su cuenta (creación asistida si hace falta). Verifica que ve el repositorio del curso. 3. **(20 min)** Recorrido guiado por el repo: estructura de carpetas `tp-u1/`, `tp-u2/`, `trabajo-final/`. Explicación de qué se entrega en cada una y cómo se crea un archivo nuevo.

Bloque 2 — Consolidación de lógica (45 min) 1. **(15 min)** Del diagrama al pseudocódigo: escribir en lenguaje natural estructurado el algoritmo del encuentro anterior (paraguas / kiosco abierto). 2. **(15 min)** Pseudocódigo con decisiones y bucles: escribir “mostrar la tabla del 5” como texto estructurado. 3. **(15 min)** Relación con Python: “Todo esto que escribimos como diagrama o pseudocódigo, lo van a escribir como código real a partir del Encuentro 4.”

Pista fortalecimiento (45 min + 45 min)

Bloque 1 — Documentación en la plataforma (45 min) 1. **(10 min)** Explicación de la organización del repo grupal y el flujo de trabajo con GitHub (issues como tareas, ramas como versiones). 2. **(15 min)** Cada estudiante verifica su cuenta de GitHub, explora el repositorio del curso y localiza los archivos de las unidades. 3. **(20 min)** Desafío: crear un archivo `bitacora.md` en su computadora con los comandos y algoritmos trabajados, 列表o en el repo como primer commit simbólico (el docente guía, sin Git todavía — se usará la interfaz web).

Bloque 2 — Algoritmos de profundización (45 min) 1. **(15 min)** Desafío de búsqueda: describir el algoritmo para encontrar el número más grande en una lista de tres números (primero en pseudocódigo, luego como diagrama). 2. **(15 min)** Desafío de validación: diseñar un algoritmo que valide si un año es bisiesto — secuencia de condiciones. Comparar con la solución real en Python (el docente la muestra como adelanto). 3. **(15 min)** Bitácora: registrar los algoritmos creados y las preguntas que surgieron, para retomarlas en la Unidad 1.

Cierre conjunto (20 min)

Plenaria final: el docente muestra la pantalla del repositorio grupal en GitHub. “Este es su espacio de trabajo para todo el año. Cada TP, cada desafío, cada entrega — todo va a estar acá.” Verifica que todos tengan sesión iniciada y puedan ver el repo. Cierra con: “El Encuentro 4 empezamos con Python — variables, tipos, `print()` y `input()` . Todo lo que practicaron estos dos encuentros les va a servir.”

Criterios de logro

Condición	Criterio
Intensificación	El estudiante navega por el sistema de archivos con comandos básicos (<code>cd</code> , <code>ls</code> , <code>mkdir</code>), dibuja diagramas de flujo simples (secuencia, decisión, repetición), e ingresa a la plataforma GitHub con su cuenta propia.
Fortalecimiento	El estudiante ejecuta comandos encadenados de terminal, documenta su aprendizaje en una bitácora personal, y expresa algoritmos de mediana complejidad (búsqueda, validación compuesta) como pseudocódigo estructurado.
Ambos grupos	Participa de las plenarias de apertura y cierre, y puede explicar con sus palabras la relación entre un diagrama de flujo, un algoritmo y un programa de computadora.

Intensificación y fortalecimiento de las Unidades 1 y 2 — Encuentros 17 y 18

Metadatos

Campo	Valor
Momento	Intensificación y fortalecimiento de las Unidades 1 y 2 — primera instancia
Encuentros	17 y 18
Duración	2 encuentros × 120 min (240 min total)
Destinatarios	Totalidad del curso, con pistas diferenciadas por condición
Requisitos	Haber cursado las unidades 1 y 2; tener resueltos o intentados los TP-U1 y TP-U2; repo grupal GitHub clonado en la PC
Lugar	Aula de informática con VS Code, terminal y repo grupal GitHub

Acuerdo pedagógico

Dimensión	Grupo de intensificación (recuperación pedagógica)	Grupo de fortalecimiento (profundización)
Contenidos	U1: tipos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>), entrada/salida (<code>print</code> , <code>input</code>), condicionales (<code>if</code> / <code>elif</code> / <code>else</code>), bucles (<code>for</code> , <code>while</code>). U2: listas (mutabilidad, slicing), tuplas y set, diccionarios, funciones (<code>def</code> , parámetros, retorno), bloque principal (<code>if __name__</code>).	U1: condicionales anidados con operadores lógicos, bucles con <code>break</code> / <code>continue</code> / <code>else</code> , comprensión de listas. U2: funciones con argumentos por nombre y valores default, funciones con retorno múltiple (tupla), manejo de excepciones (<code>try</code> / <code>except</code> básico), enumeración con <code>enumerate</code> .
Actividad / metodología	Ejercicios guiados paso a paso: corregir errores comunes, refactorizar programas existentes del TP, cerrar el ciclo Git (commit + push) del TP correspondiente.	Desafíos de profundización autónomos: extender programas con nuevas funcionalidades, refactorizar con comprensión de listas y funciones de orden superior.
Recursos	VS Code, terminal, repo grupal clonado, guía impresa de núcleos U1 y U2, ejemplos resueltos del TP-U1 y TP-U2, lista de verificación de objetivos por alumno.	VS Code, terminal, repo grupal, consignas de desafío impresas, documentación oficial de Python (local o descargada).

Desarrollo del Encuentro 17

Apertura conjunta (10 min)

Plenaria: el docente recorre los núcleos de U1 y U2 con un mapa conceptual en el pizarrón (tipos → variables → entrada/salida → condicionales → bucles → colecciones → funciones). Se asigna cada estudiante a su pista según el desempeño registrado en los TP-U1 y TP-U2. **No se permite celular.**

Pista intensificación (45 min + 45 min)

Bloque 1 — U1: tipos, condicionales y bucles (45 min) 1. (15 min) Repaso declaración de variables y tipos. Ejercicio

guiado: programa que pida nombre, edad y promedio, los muestre con formato. Errores comunes: no convertir con `int()` / `float()`, concatenar tipos distintos. 2. **(15 min)** Condicionales. Ejercicio: programa para un kiosco escolar que determine si un cliente ((cliente)) puede comprar un producto ((producto)) según su edad y el tipo de producto. Usar `if/elif/else`. 3. **(15 min)** Bucles. Ejercicio: mostrar un menú con opciones (1: ver producto, 2: calcular total, 3: salir) usando `while` y `break`.

Bloque 2 — U2: listas y tuplas (45 min) 1. **(15 min)** Listas: mutabilidad, índice, `append`, `remove`, slicing. Ejercicio: gestionar la lista de socios ((socios)) del club de barrio — agregar, eliminar, mostrar. 2. **(15 min)** Tuplas y set: inmutabilidad, desempacado, operaciones de conjunto (unión, intersección). Ejercicio: separar socios ((socios)) activos de inactivos usando set. 3. **(15 min)** Mini-integrador: combinar lista de socios ((memberList)) con bucle `for` para mostrar los que tienen cuota ((clubFee)) al día. Commit y push en el repo.

Pista fortalecimiento (45 min + 45 min)

Bloque 1 — U1 profundizado (45 min) 1. **(15 min)** Condicionales anidados con `and` / `or` / `not`. Desafío: validar si un socio ((socio)) del club puede acceder al salón de usos múltiples según edad, cuota al día ((clubFee)) y autorización. 2. **(15 min)** Bucles con `continue` y `else`. Desafío: procesar la lista de socios ((memberList)) y saltar los que tienen cuota vencida ((vencimiento)), mostrar solo los activos ((activeMembers)). 3. **(15 min)** Comprensión de listas. Desafío: generar una lista de edades ((memberAge)) filtrada (solo mayores de 18) en una línea.

Bloque 2 — U2 profundizado (45 min) 1. **(15 min)** Funciones con valores default y argumentos por nombre. Desafío: función `findMember(memberList, nombre, activo=True)` que busque socios ((socios)) activos o inactivos según el parámetro. 2. **(15 min)** Retorno múltiple. Desafío: función que reciba la lista de socios ((memberList)) y devuelva el más joven y el más viejo como tupla. 3. **(15 min)** Enumeración con `enumerate`. Desafío: mostrar la lista de socios ((socios)) numerada para un menú interactivo.

Cierre conjunto (20 min)

Puesta en común: cada pista comparte la línea de código que más les costó resolver. El docente destaca los errores típicos (olvidar `int()`, confundir `=` con `==`, modificar una lista mientras se la recorre). Anticipa que en el encuentro siguiente se completa U2 con diccionarios y funciones, y se consolida el ciclo Git.

Desarrollo del Encuentro 18

Apertura conjunta (10 min)

Repaso relámpago: el docente escribe en vivo un programa que pida tres números, los guarde en una lista y muestre el promedio. Pregunta “¿qué falta? — diccionarios y funciones”. Anuncia que hoy se consolidan esos dos núcleos y se cierra el ciclo Git de los TP-U1 y TP-U2.

Pista intensificación (45 min + 45 min)

Bloque 1 — Diccionarios (45 min) 1. **(15 min)** Concepto: pares clave-valor. Ejercicio: crear un diccionario con los datos de un socio ((socio)) del club (nombre, edad, cuota, vencimiento) y acceder a cada campo. 2. **(15 min)** Recorrido de diccionarios con `for`. Ejercicio: mostrar el listado completo de socios ((socios)) almacenados como diccionarios dentro de una lista. 3. **(15 min)** Mini-integrador: buscar un socio ((findMember)) por nombre en una lista de diccionarios. Devolver todos sus datos o un mensaje “no encontrado”.

Bloque 2 — Funciones y bloque principal (45 min) 1. **(15 min)** Definir función con `def`, parámetros y `return`. Ejercicio: función `calcular_cuota(monto_base, descuento)` que devuelva el total. 2. **(15 min)** Función que reciba una lista de socios

((memberList)) y devuelva los activos ((activeMembers)). Implementar y probar desde `main()` . 3. **(15 min)** Bloque principal (`if __name__ == "__main__"`). Refactorizar el programa del encuentro 17 para que use funciones y bloque principal. Commit y push.

Pista fortalecimiento (45 min + 45 min)

Bloque 1 — Diccionarios profundizados (45 min) 1. **(15 min)** Diccionarios anidados. Desafío: modelar un club ((club)) con lista de socios ((socios)) donde cada socio tenga datos personales y un historial de pagos (lista de fechas). 2. **(15 min)** Métodos de diccionarios (`get` , `keys` , `values` , `items`). Desafío: generar un reporte con los socios ((socios)) que adeudan más de 2 cuotas ((clubFee)). 3. **(15 min)** Comprensión de diccionarios. Desafío: crear un diccionario {nombre: edad} a partir de una lista de diccionarios de socios ((socios)) en una línea.

Bloque 2 — Funciones profundizadas (45 min) 1. **(15 min)** Funciones como valores de primera clase. Desafío: pasar una función como argumento a otra función (callback) para ordenar la lista de socios ((memberList)) por distintos criterios. 2. **(15 min)** `lambda` y `map / filter` . Desafío: filtrar socios ((socios)) mayores de 18 con `filter` y una función lambda. 3. **(15 min)** `try / except` básico. Desafío: envolver la entrada de datos en un bloque `try / except` para validar que la edad y la cuota ((clubFee)) sean numéricas.

Cierre conjunto (20 min)

Plenaria final: el docente muestra el repo grupal actualizado con los commits del encuentro. Verifica que los TP-U1 y TP-U2 estén commiteados y pusheados. “Con esto cierran las dos primeras unidades. En el proyecto puente (encuentros 19 y 20) van a integrar todo: entrada, condicionales, bucles, listas, diccionarios, funciones — un solo programa que vale como recuperatorio.”

Criterios de logro

Condición	Criterio
Intensificación	El estudiante escribe programas que usan tipos correctos, <code>input()</code> / <code>print()</code> , condicionales, bucles, listas, tuplas, diccionarios y funciones con <code>def</code> y <code>return</code> . Utiliza el bloque principal (<code>if __name__</code>). Resuelve al menos 3 de los 4 bloques de cada encuentro. Commitea y pushea al repo grupal.
Fortalecimiento	El estudiante compone condicionales anidados con operadores lógicos, usa comprensión de listas y diccionarios, escribe funciones con argumentos por nombre y valores default, y emplea <code>lambda</code> , <code>map / filter</code> y <code>try / except</code> básico. Resuelve los desafíos completos de ambos encuentros.
Ambos grupos	Participa de las plenarias de apertura y cierre, y actualiza el repositorio grupal con los avances de cada encuentro.

Proyecto puente integrador de las Unidades 1 y 2 — Encuentros 19 y 20

Metadatos

Campo	Valor
Momento	Proyecto puente integrador de las Unidades 1 y 2
Encuentros	19 y 20
Duración	2 encuentros × 120 min (240 min total)
Destinatarios	Totalidad del curso (una sola pista)
Requisitos	Haber completado los TP-U1 y TP-U2; tener el repo grupal clonado con los dos TP commiteados; conocer variables, tipos, entrada/salida, condicionales, bucles, listas, diccionarios y funciones en Python
Lugar	Aula de informática con VS Code, terminal y repo grupal GitHub

Acuerdo pedagógico

Dimensión	Única pista para todo el curso
Contenidos	Proyecto integrador (programa de consola) que articula: entrada/salida (<code>print</code> , <code>input</code>), condicionales (<code>if</code> / <code>elif</code> / <code>else</code>), bucles (<code>while</code> / <code>for</code>), listas, diccionarios y funciones (<code>def</code> , parámetros, retorno). Dominio: sistema de gestión de un club de barrio (socios, cuotas, vencimientos).
Actividad / metodología	Construcción progresiva del programa (de consola, sin persistencia). Encuentro 19: consigna, planificación grupal y primera parte con commits. Encuentro 20: finalización, puesta en común y cierre con defensa. Trabajo individual con revisión entre pares.
Recursos	VS Code, terminal, repo grupal clonado, consigna del proyecto puente impresa con rúbrica de 100 puntos, ejemplos resueltos de U1 y U2.

Criterios de evaluación — rúbrica (100 puntos)

Dimensión	Puntaje	Indicadores
Entrada y condicionales	20 pts	Usa <code>input()</code> y <code>int()</code> / <code>float()</code> correctamente. Aplica <code>if</code> / <code>elif</code> / <code>else</code> para decidir flujo del menú y validar datos.
Bucles	15 pts	Usa <code>while</code> para el menú principal (hasta que el usuario elija salir). Usa <code>for</code> para recorrer listas y diccionarios.
Listas y diccionarios	25 pts	Almacena socios como diccionarios dentro de una lista. Opera sobre la lista (agregar, eliminar, buscar, listar).
Funciones y bloque principal	20 pts	Define funciones con <code>def</code> , parámetros y <code>return</code> . Organiza el programa con <code>if __name__ == "__main__":</code> .
Calidad del código	10 pts	Nombres descriptivos, comentarios útiles, código sin líneas muertas, entrada de datos validada mínimamente.
Entrega y defensa	10 pts	Repositorio GitHub actualizado con commit del proyecto. Breve explicación oral del programa y las decisiones tomadas.

Desarrollo del Encuentro 19

Apertura (10 min)

El docente presenta el proyecto puente: “Van a construir un programa de consola para gestionar los socios de un club de barrio. No es un TP nuevo: es integrar todo lo que ya saben de U1 y U2 en un solo programa. No usamos archivos ni base de datos — todo se hace en memoria.” **No se permite celular.** Se entrega la consigna impresa y se explica la rúbrica de 100 puntos.

Desarrollo (45 min + 45 min)

Bloque 1 — Consigna y planificación grupal (45 min)

- (10 min)** Lectura colectiva de la consigna. El docente aclara el alcance: el programa debe ofrecer un menú con las opciones:
 - 1: Agregar socio ((socio))
 - 2: Listar todos los socios ((socios))
 - 3: Buscar socio por nombre ((findMember))
 - 4: Mostrar socios activos ((activeMembers)), es decir, con cuota al día
 - 5: Mostrar socios con cuota vencida
 - 6: Salir
- (15 min)** Planificación en parejas: cada estudiante diagrama la estructura del programa (funciones necesarias, lista de diccionarios como estructura de datos, flujo del menú).
- (20 min)** Puesta en común de los diagramas. El docente valida y sugiere nombres de funciones: `agregar_socio()`, `listar_socios()`, `buscar_socio()`, `socios_activos()`, `socios_vencidos()`. Se define la estructura de datos: cada socio es un

diccionario con claves "nombre", "edad" (memberAge), "cuota" (clubFee), "vencimiento" (mes como string).

Bloque 2 — Primera parte: estructura base y opciones 1 y 2 (45 min)

1. **(15 min)** Implementar opción 1 (agregar socio): pedir nombre, edad, cuota ((clubFee)) y vencimiento por `input()`. Crear el diccionario del socio ((socio)) y agregarlo a la lista global. Validar que edad y cuota sean números.
2. **(15 min)** Implementar opción 2 (listar socios): recorrer la lista de socios ((socios)) con `for` y mostrar cada uno con formato.
3. **(15 min)** Probar ambas opciones. Commit y push al repo grupal con mensaje "feat: agregar socio y listar socios".

Cierre (20 min)

Puesta en común de pantallas: el docente verifica que cada estudiante tenga las opciones 1 y 2 funcionando. Revisa commits en GitHub. Anticipa las opciones del próximo encuentro (3, 4 y 5) y recuerda que la opción 6 (salir) debe cerrar el bucle `while` con `break`.

Desarrollo del Encuentro 20

Apertura (10 min)

Repaso de lo creado en el encuentro anterior. El docente muestra una versión completa del programa en el proyector y recorre las opciones que faltan. Recuerda que hoy se finaliza, se hace la defensa y se entrega con la rúbrica completa. **No se permite celular.**

Desarrollo (45 min + 45 min)

Bloque 1 — Opciones 3, 4 y 5 (45 min)

1. **(15 min)** Opción 3 (buscar socio por nombre): implementar función (findMember) que recorra la lista de diccionarios con un bucle `for` y compare el nombre ingresado con el campo "nombre" de cada socio ((socio)). Si no encuentra, mostrar mensaje "Socio no encontrado".
2. **(15 min)** Opción 4 (socios activos): implementar función (activeMembers) que filtre la lista por cuota al día (comparar vencimiento con el mes actual ingresado al iniciar el programa). Mostrar solo los cumplidores.
3. **(15 min)** Opción 5 (socios con cuota vencida): función análoga que muestre los que tienen vencimiento anterior al mes actual. Probar todo el ciclo del menú.

Bloque 2 — Bloque principal, pulido y entrega (45 min)

1. **(15 min)** Refactorizar: envolver todo el programa en funciones y agregar `if __name__ == "__main__"`. Mover la lista de socios como variable global o pasarla como parámetro.
2. **(15 min)** Pulido: verificar nombres de variables, agregar comentarios donde la lógica no sea obvia, validar que la entrada del menú sea numérica (con `try / except` básico).
3. **(15 min)** Commit final ("feat: proyecto puente completo") y push al repo grupal. Cada estudiante verifica que GitHub muestre el archivo `.py` correctamente.

Cierre con defensa (20 min)

El docente recorre los puestos y cada estudiante explica brevemente: - Qué hace el programa - Cómo eligió las estructuras de datos (por qué diccionarios dentro de una lista) - Una función que le haya costado y cómo la resolvió

El docente registra los puntajes parciales en la rúbrica de 100 puntos. Entrega el resultado al final del encuentro.

Criterios de logro

Condición	Criterio
Aprobado (≥ 60 pts)	El programa compila y ejecuta. Menú funcional con al menos opciones 1, 2, 3 y 6. Usa diccionarios, listas, funciones y bloque principal. Hay commit en el repo.
Destacado (≥ 85 pts)	Cumple el criterio anterior + completa las opciones 4 y 5 con lógica correcta de vencimiento. El código está bien comentado, las funciones son reutilizables, y la defensa muestra comprensión de las decisiones de diseño.
No aprobado (< 60 pts)	El programa no compila, no corre, o falta funcionalidad central (no usa funciones, no usa diccionarios, no hay commit). El estudiante pasa al grupo de intensificación en los momentos siguientes (encuentros 34-35 o diciembre).

Intensificación y fortalecimiento de las Unidades 3 y 4 — Encuentros 34 y 35

Metadatos

Campo	Valor
Momento	Intensificación y fortalecimiento de las Unidades 3 y 4
Encuentros	34 y 35
Duración	2 encuentros × 120 min (240 min total)
Destinatarios	Totalidad del curso, con pistas diferenciadas por condición
Requisitos	Haber cursado las unidades 3 y 4; tener TP-U3 commiteado y trabajo final en progreso o pendiente; repo grupal GitHub clonado
Lugar	Aula de informática con VS Code, terminal y repo grupal GitHub

Acuerdo pedagógico

Dimensión	Grupo de intensificación (recuperación pedagógica)	Grupo de fortalecimiento (profundización)
Contenidos	U3: métodos de cadenas (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>), f-strings, validación con <code>try / except</code> , menú en memoria (sin archivos). U4: README profesional, issues, ramas, PR, main protegida. Quienes adeuden el trabajo final lo completan con commits y push.	U3: composición de métodos de cadenas para parseo complejo, validación robusta con <code>try / except / else / finally</code> , menú con sub-menús. U4: resolución de issues mediante ramas, PR con revisión cruzada entre pares, README avanzado con badges.
Actividad / metodología	Ejercicios guiados: procesar pedidos de una pizzería (strings, validación), construir menú en memoria, completar y cerrar el trabajo final con README y PR.	Desafíos autónomos: parser de texto de pedidos con formato libre, validador de datos de entrada, menú jerárquico. Flujo profesional completo: issue → rama → PR → merge.
Recursos	VS Code, terminal, repo grupal clonado, guía impresa de métodos de cadenas y <code>try / except</code> , plantilla README, lista de verificación de objetivos por alumno.	VS Code, terminal, repo grupal, consignas de desafío impresas, documentación oficial de Python (local o descargada), plantilla README avanzado.

Desarrollo del Encuentro 34

Apertura conjunta (10 min)

Plenaria: el docente recorre los núcleos de U3 (procesamiento de texto, validación, menú en memoria) y U4 (flujo profesional). Explica que hoy se enfocan en U3 y que el trabajo final se retoma en el Encuentro 35. Se asigna cada estudiante a su pista según el desempeño en TP-U3 y el estado del trabajo final. **No se permite celular.**

Pista intensificación (45 min + 45 min)

Bloque 1 — Métodos de cadenas (45 min) 1. (15 min) Repaso: `split()` , `strip()` , `join()` , `replace()` . Ejercicio guiado: dividir un pedido ((pedido)) de pizzería en ingredientes ((ingredientes)), limpiar espacios, unir con comas. 2. **(15 min)** f-strings:

formato de salida con variables y expresiones. Ejercicio: mostrar el detalle del pedido de la pizzería ((pizzería)) con formato legible: nombre del comprador ((comprador)), ingredientes ((ingredientes)), precio ((orderPrice)). 3. **(15 min)** Aplicación: procesar una línea de texto con datos de un pedido ((orderText)) separados por `|`, extraer nombre, teléfono ((telefono)) e ingredientes, y mostrar todo con f-strings.

Bloque 2 — Validación y menú en memoria (45 min) 1. **(15 min)** `try/except` básico. Ejercicio: capturar `ValueError` al convertir precios ((orderPrice)) desde string. Mostrar mensaje de error amigable sin que el programa se rompa. 2. **(15 min)** Validación de teléfono ((validPhone)): función que verifique que un string tenga solo dígitos y 10 caracteres usando `isdigit()` y `len()`. Integrar en el programa de pedidos. 3. **(15 min)** Menú en memoria: crear un menú con opciones (1: cargar pedido, 2: ver pedidos, 3: buscar por nombre, 4: salir) usando un diccionario como almacenamiento en memoria. Sin archivos, sin base de datos.

Pista fortalecimiento (45 min + 45 min)

Bloque 1 — Parseo complejo de texto (45 min) 1. **(15 min)** Composiciones de métodos de cadena: procesar una línea como `"Muzzarella;Tomate;Aceitunas (sin cebolla)"` extrayendo ingredientes base y extras con `split()` múltiple. 2. **(15 min)** Expresiones regulares básicas con `re.search` (mención, no profundización): validar formato de teléfono ((telefono)) con patrón simple. Comparar con la solución de `isdigit()/len()`. 3. **(15 min)** Parseo de texto libre: procesar un pedido ((orderText)) escrito en lenguaje natural ("2 muzza + 1 napo + 1 fainá") y extraer cantidades, variedades y precios estimados.

Bloque 2 — Validación robusta y sub-menús (45 min) 1. **(15 min)** `try/except/else/finally`: estructura completa. Verificar que el bloque `else` se ejecuta solo si no hubo excepción y `finally` siempre. 2. **(15 min)** Función `validPhone` ((validPhone)) y `validName` ((validName)): validar nombre del cliente ((customerName)) que contenga al menos nombre y apellido, y teléfono de exactamente 10 dígitos. 3. **(15 min)** Menú con sub-menús: opción "Pedidos" que abra un segundo nivel (1: cargar, 2: modificar, 3: eliminar, 4: volver). Implementar con diccionario de funciones.

Cierre conjunto (20 min)

Puesta en común: cada pista comparte un fragmento de código que resolvió. El docente destaca la diferencia entre usar `split()` simple y parsear texto libre. Anticipa que en el próximo encuentro se cierra U4 con el flujo profesional (README, issues, ramas, PR) y se completa el trabajo final quienes lo adeuden.

Desarrollo del Encuentro 35

Apertura conjunta (10 min)

Repaso: el docente muestra en vivo un programa de pedidos de pizzería funcionando con menú en memoria. Pregunta: "Esto funciona en mi PC. ¿Cómo lo comparto con el mundo?". Respuesta: con README, issues, ramas y PR. Hoy cerramos U4.

Pista intensificación (45 min + 45 min)

Bloque 1 — README profesional y cierre del repo (45 min) 1. **(15 min)** ¿Qué es un README? Estructura básica: título, descripción, tecnologías (Python 3.x), cómo ejecutar, ejemplo de uso. Plantilla provista. 2. **(15 min)** Cada estudiante crea o actualiza el README del repo grupal con la plantilla. Lo commitea en una rama `docs/readme`. 3. **(15 min)** Issues: crear un issue en GitHub con la descripción de una funcionalidad faltante. Asignarla a un compañero.

Bloque 2 — Trabajo final: commits, PR y main protegida (45 min) 1. **(20 min)** Quienes adeudan el trabajo final: conectarlo al repo grupal. Commit del código faltante en una rama `feature/trabajo-final`. Abrir PR hacia `main`. 2. **(15 min)** El docente verifica que main esté protegida (no se puede pushear directo). El PR debe ser revisado y mergeado por otro compañero o el

docente. 3. **(10 min)** Quienes ya completaron el trabajo final: ayudar a un compañero con su PR (code review cruzado). Verificar que el PR tenga al menos un comentario de revisión.

Pista fortalecimiento (45 min + 45 min)

Bloque 1 — README avanzado y flujo profesional (45 min)1. **(15 min)** README avanzado: badges de estado, tabla de contenidos, sección de contribución, licencia. Generar badges con shields.io. 2. **(15 min)** Cada estudiante mejora el README del repo con badges y tabla de contenidos. Crea un issue de mejora y lo resuelve con una rama y PR. 3.**(15 min)** Code review cruzado: revisar el PR de un compañero, dejar al menos un comentario sustantivo (sugerencia de mejora, pregunta sobre la implementación).

Bloque 2 — Resolución de issues con ramas y PR (45 min)1. **(15 min)** Tomar un issue abierto del repo (propio o de un compañero), crear rama con nombre descriptivo (`fix/memoria-pedidos` , `feat/ordenar-menu`). 2. **(15 min)** Implementar la solución, commit con mensaje que referencie el issue (“Closes #3: agrega ordenamiento alfabético de pedidos”). Push, abrir PR. 3.**(15 min)** Mergear el PR tras revisión. Verificar que main se actualizó. Eliminar la rama de feature.

Cierre conjunto (20 min)

Plenaria final: el docente proyecta el repo grupal y muestra los PRs mergeados, los issues resueltos, el README actualizado y la main protegida. “Esto es un repositorio profesional. Lo que vieron hoy es cómo se trabaja en equipo en cualquier empresa de software. Cerramos U4.”

Criterios de logro

Condición	Criterio
Intensificación	El estudiante procesa strings con <code>split</code> / <code>strip</code> / <code>join</code> / <code>replace</code> , escribe f-strings, valida entrada con <code>try</code> / <code>except</code> , construye un menú en memoria con diccionarios, y crea/actualiza el README del repo. Completa el trabajo final con commits, rama y PR. Resuelve al menos 3 de los 4 bloques.
Fortalecimiento	El estudiante compone métodos de cadenas para parseo complejo, aplica <code>try</code> / <code>except</code> / <code>else</code> / <code>finally</code> , implementa sub-menús, lidera code review cruzado, y resuelve issues con ramas y PRs. README avanzado con badges.
Ambos grupos	Participa de las plenarias de apertura y cierre, actualiza el repositorio grupal, y completa el cierre del trabajo final con commits y PR.

Intensificación de diciembre — Camino mínimo completo

Metadatos

Campo	Valor
Momento	Intensificación de diciembre (fuera de la estructura anual)
Duración	2 encuentros × 120 min (240 min total)
Destinatarios	Estudiantes que no alcanzaron los objetivos mínimos del curso durante el ciclo lectivo
Requisitos	Haber cursado la totalidad del año; tener cuenta de GitHub activa y repo grupal clonado; asistencia obligatoria
Lugar	Aula de informática con VS Code, terminal y repo grupal GitHub
Evaluación	Camino mínimo completo del curso. Criterio: Apto / No apto aún por objetivo mínimo . No hay puntaje numérico.

Acuerdo pedagógico

Dimensión	Grupo de intensificación (recuperación pedagógica) — único grupo
Contenidos	Camino mínimo completo del curso: 1) entrada/salida con <code>print()</code> / <code>input()</code> ; 2) variables y tipos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>); 3) condicionales (<code>if</code> / <code>elif</code> / <code>else</code>); 4) bucles (<code>for</code> , <code>while</code>); 5) listas y diccionarios; 6) funciones con <code>def</code> , parámetros y <code>return</code> ; 7) bloque principal (<code>if __name__ == "__main__"</code>); 8) métodos de cadenas (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>), f-strings; 9) validación con <code>try</code> / <code>except</code> ; 10) menú en memoria; 11) entregas por GitHub del repo grupal (carpetas <code>tp-u1/</code> a <code>trabajo-final/</code>); 12) defensa oral del programa presentado. Dominio: verdulería (productos, kilos, precio, factura, stock).
Actividad / metodología	Recorrido acelerado por los contenidos mínimos del año con énfasis en los objetivos no alcanzados. Cada estudiante reconstruye —o completa— un programa integrador de consola sobre el dominio de la verdulería, dividido en dos encuentros. Encuentro 1: entrada, condicionales, bucles, listas, diccionarios. Encuentro 2: funciones, validación, menú, entregas GitHub y defensa.
Recursos	VS Code, terminal, repo grupal clonado, guía paso a paso del camino mínimo (impresa), lista de verificación de objetivos por alumno, plantilla README.

Desarrollo del Encuentro 1

Apertura (10 min)

El docente explica la modalidad de la instancia de diciembre: “No es un curso nuevo — es la oportunidad de demostrar que pueden hacer el camino mínimo. Cada objetivo que logren hoy es un objetivo aprobado.” Se entrega la lista de verificación individual con los objetivos no alcanzados de cada alumno. Sale del ciclo reposo para diciembre. **No se permite celular.**

Desarrollo (45 min + 45 min)

Bloque 1 — Entrada, tipos y condicionales (45 min)

1. **(15 min)** Ejercicio 1 — programa base: crear un archivo `verduleria.py`. Declarar variables con los tipos correctos para representar un producto ((producto)) de la verdulería: nombre (str), precio (float, (productPrice)), kilos disponibles (float, (stockWeight)). Pedir datos con `input()` y convertir. Mostrar con f-string.
2. **(15 min)** Ejercicio 2 — condicionales: agregar validación. Si el producto tiene menos de 1 kilo ((kilos)) disponible, mostrar "Stock bajo". Si el precio supera los \$500, aplicar un descuento del 10%. Usar `if / elif / else`.
3. **(15 min)** Ejercicio 3 — bucle `while`: menú simple con opciones (1: ver producto, 2: vender, 3: salir). Usar `while True` y `break`. Cada opción imprime el estado actual.

Bloque 2 — Listas, diccionarios y bucles (45 min)

1. **(15 min)** Ejercicio 4 — lista de productos ((productList)): almacenar productos como diccionarios dentro de una lista. Pedir datos por teclado y agregar con `append()`.
2. **(15 min)** Ejercicio 5 — mostrar productos con `for`: recorrer la lista de productos ((productList)) y mostrar cada uno con f-string formateado.
3. **(15 min)** Ejercicio 6 — buscar producto ((productName)) por nombre: recorrer la lista con `for` y comparar. Mostrar el producto encontrado o "Producto no encontrado".

Cierre (20 min)

Verificación individual: el docente recorre los puestos y marca en la lista de verificación qué objetivos están logrados (1-6) y cuáles quedan pendientes para el encuentro siguiente. Cada estudiante hace commit del avance con mensaje `"feat: verduleria base y listado"`.

Desarrollo del Encuentro 2

Apertura (10 min)

Repaso de lo logrado en el encuentro anterior. El docente presenta los objetivos del día: funciones, validación, menú completo, entrega en GitHub y defensa.

Desarrollo (45 min + 45 min)

Bloque 1 — Funciones y validación (45 min)

1. **(15 min)** Ejercicio 7 — funciones: refactorizar el programa anterior. Extraer cada operación a una función: `agregar_producto()`, `listar_productos()`, `buscar_producto()` ((productName)), `vender_producto()` ((sellProduct)).
2. **(15 min)** Ejercicio 8 — bloque principal: agregar `if __name__ == "__main__"` y dentro el menú principal que llama a las funciones. Verificar que el programa funcione igual que antes.
3. **(15 min)** Ejercicio 9 — validación con `try / except`: envolver la conversión de precio ((productPrice)) y kilos ((kilos)) en un bloque `try / except` que capte `ValueError`. Mostrar "Dato inválido, intente de nuevo" y pedir el dato otra vez.

Bloque 2 — Menú, entrega y defensa (45 min)

1. **(15 min)** Ejercicio 10 — menú completo en memoria: opciones (1: agregar producto, 2: listar productos, 3: buscar producto, 4: vender producto, 5: factura ((factura)), 6: salir). Factura muestra el resumen de la venta actual.

2. **(15 min)** Ejercicio 11 — entregas GitHub: commit final con mensaje `"feat: verduleria completa camino mínimo"`. Crear carpeta `trabajo-final/` en el repo grupal si no existe. Pushear. Verificar que GitHub muestra el archivo.
3. **(15 min)** Ejercicio 12 — README básico: crear o actualizar `README.md` en la raíz del repo con título, descripción ("Sistema de gestión de verdulería en Python"), tecnologías y cómo ejecutar.

Cierre con defensa (20 min)

El docente recorre los puestos con la lista de verificación. Cada estudiante explica brevemente: - Qué hace el programa - Cómo valida los datos de entrada - Una función que le haya costado implementar

Si el estudiante cumple todos los objetivos (1 a 12), recibe **Apto**. Si falta alguno, se registra como **No apto aún por objetivo mínimo** y se detalla cuál(es) objetivo(s) no alcanzó, junto con la sugerencia de la instancia de marzo como siguiente oportunidad con el mismo estándar.

Evaluación

Resultado	Condición
Apto	El programa compila y ejecuta. Usa <code>input()</code> / <code>print()</code> , condicionales, bucles, listas, diccionarios, funciones, bloque principal, <code>try</code> / <code>except</code> , f-strings y menú en memoria. El repo tiene commit y README. El estudiante explica su código durante la defensa.
No apto aún por objetivo mínimo	Falta al menos uno de los elementos anteriores. Se especifica qué objetivo(s) no se alcanzó y se sugiere la instancia de marzo como siguiente oportunidad con el mismo estándar.

Criterios de logro

Condición	Criterio
Apto	El estudiante construye un programa de consola completo (entrada validada, control de flujo, colecciones, funciones, menú en memoria) sobre el dominio de la verdulería. Entrega el código en GitHub con README. Durante la defensa explica el funcionamiento general y la lógica de al menos una función.
No apto	El estudiante no completa alguno de los componentes anteriores o no puede explicar el funcionamiento de su propio código. Recibe la lista de objetivos pendientes y la fecha de la instancia de marzo.

Intensificación de marzo — Camino mínimo completo

Metadatos

Campo	Valor
Momento	Intensificación de marzo (fuera de la estructura anual, previa al nuevo ciclo)
Duración	2 encuentros × 120 min (240 min total)
Destinatarios	Estudiantes que no alcanzaron los objetivos mínimos en la instancia de diciembre
Requisitos	Haber cursado la totalidad del año y la instancia de diciembre; contar con cuenta de GitHub activa y repo grupal clonado; asistencia obligatoria
Lugar	Aula de informática con VS Code, terminal y repo grupal GitHub
Evaluación	Camino mínimo completo del curso (mismo estándar que diciembre). Criterio: Apto / No apto aún por objetivo mínimo . No baja el estándar; cambia el tiempo de preparación del estudiante (diciembre → marzo).

Acuerdo pedagógico

Dimensión	Grupo de intensificación (recuperación pedagógica) — único grupo
Contenidos	Camino mínimo completo del curso (idéntico al de diciembre): 1) entrada/salida con <code>print()</code> / <code>input()</code> ; 2) variables y tipos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>); 3) condicionales (<code>if</code> / <code>elif</code> / <code>else</code>); 4) bucles (<code>for</code> , <code>while</code>); 5) listas y diccionarios; 6) funciones con <code>def</code> , parámetros y <code>return</code> ; 7) bloque principal (<code>if __name__ == "__main__"</code>); 8) métodos de cadenas (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>), f-strings; 9) validación con <code>try</code> / <code>except</code> ; 10) menú en memoria; 11) entregas por GitHub del repo grupal (carpetas <code>tp-u1/</code> a <code>trabajo-final/</code>); 12) defensa oral del programa presentado. Dominio: verdulería (producto, kilos, precio, factura, stock).
Actividad / metodología	El estudiante tuvo más tiempo para prepararse (diciembre a marzo). La instancia presencial se organiza como taller de verificación: el docente revisa el programa que el estudiante trae preparado y asiste en los puntos que todavía presenten dificultad. Encuentro 1: verificación de objetivos 1 a 6 (entrada, condicionales, bucles, listas, diccionarios). Encuentro 2: verificación de objetivos 7 a 12 (funciones, validación, menú, entrega y defensa).
Recursos	VS Code, terminal, repo grupal clonado, guía paso a paso del camino mínimo (impresa desde diciembre), lista de verificación de objetivos por alumno con el registro de diciembre, plantilla README.

Desarrollo del Encuentro 1

Apertura (10 min)

El docente da la bienvenida a la instancia de marzo. Explica que el estándar es el mismo que en diciembre: “No bajamos la exigencia — ustedes tuvieron más tiempo para preparar cada objetivo. Hoy verificamos lo que traen.” Cada estudiante recibe su lista de verificación individual actualizada con los objetivos pendientes desde diciembre. **No se permite celular.**

Desarrollo (45 min + 45 min)

Bloque 1 — Verificación de entrada, tipos y condicionales (45 min)

1. **(15 min)** Verificación 1 — programa base: el estudiante muestra su archivo `verduleria.py`. El docente revisa que declare variables correctamente, use `input()` con conversión de tipos y muestre con f-strings. Marca objetivo como logrado en la lista.
2. **(15 min)** Verificación 2 — condicionales: mostrar la validación de stock ((stockWeight)) bajo y descuento por precio alto. Si no está implementado, se guía su creación en vivo.
3. **(15 min)** Verificación 3 — bucle `while` con menú simple. El estudiante ejecuta el programa y muestra las opciones 1 a 3 funcionando. Si falta, se completa con asistencia.

Bloque 2 — Verificación de listas, diccionarios y bucles (45 min)

1. **(15 min)** Verificación 4 — lista de productos ((productList)): el estudiante muestra cómo agrega productos como diccionarios a la lista. El docente revisa que la estructura de datos sea correcta (diccionarios con claves consistentes).
2. **(15 min)** Verificación 5 — mostrar productos con `for`: el estudiante ejecuta la opción de listar. Se verifica formato y que recorra toda la lista.
3. **(15 min)** Verificación 6 — buscar producto ((productName)): mostrar la función de búsqueda con resultados existentes y el mensaje “Producto no encontrado” para un nombre inexistente.

Cierre (20 min)

El docente registra los objetivos logrados en el encuentro. Si un estudiante completó todos los objetivos 1 a 6, recibe el pase al encuentro 2 para funciones, validación y entrega. Si no, se le asigna trabajo adicional para el segundo encuentro. Cada estudiante hace commit si hubo modificaciones.

Desarrollo del Encuentro 2

Apertura (10 min)

Repaso de los objetivos de funciones, validación, menú completo y entrega. El docente muestra en vivo la estructura esperada del programa completo y recuerda que el README y la defensa son parte de la evaluación.

Desarrollo (45 min + 45 min)

Bloque 1 — Verificación de funciones, bloque principal y validación (45 min)

1. **(15 min)** Verificación 7 — funciones: el estudiante muestra las funciones `agregar_producto()`, `listar_productos()`, `buscar_producto()` ((productName)) y `vender_producto()` ((sellProduct)). El docente verifica que tengan parámetros y retorno donde corresponda.
2. **(15 min)** Verificación 8 — bloque principal: el estudiante muestra `if __name__ == "__main__"` y que el programa funcione igual que en el encuentro anterior.

3. **(15 min)** Verificación 9 — `try / except` : el estudiante ingresa un dato no numérico en precio ((productPrice)) o kilos ((kilos)) y muestra que el programa captura el error sin romperse.

Bloque 2 — Verificación de menú completo, entrega GitHub y defensa (45 min)

1. **(15 min)** Verificación 10 — menú completo: opciones 1 a 6 funcionando (agregar, listar, buscar, vender, factura, salir). La factura ((factura)) muestra los totales de la venta.
2. **(15 min)** Verificación 11 — entrega GitHub: el estudiante muestra el commit final (`"feat: verduleria completa camino mínimo"`) en el repo grupal dentro de la carpeta `trabajo-final/` . Verificar que GitHub muestre el archivo.
3. **(15 min)** Verificación 12 — README: mostrar el `README.md` con título, descripción, tecnologías y cómo ejecutar. Si falta, se completa en el encuentro.

Cierre con defensa (20 min)

El docente recorre los puestos con la lista de verificación. Cada estudiante explica brevemente: - Qué hace el programa - Cómo valida los datos de entrada - La diferencia entre una lista y un diccionario en su programa - Una función que le haya costado implementar

Si el estudiante cumple todos los objetivos (1 a 12), recibe **Apto**. Si falta alguno, se registra como **No apto aún por objetivo mínimo** y se detalla cuáles objetivos no alcanzó, junto con la recomendación de recurrar la materia si el período lo permite.

Evaluación

Resultado	Condición
Apto	El programa compila y ejecuta. Usa <code>input()</code> / <code>print()</code> , condicionales, bucles, listas, diccionarios, funciones, bloque principal, <code>try / except</code> , f-strings y menú en memoria. El repo tiene commit y README. El estudiante explica su código durante la defensa.
No apto aún por objetivo mínimo	Falta al menos uno de los elementos anteriores. Se especifica qué objetivo(s) no se alcanzó y se informa al estudiante junto con la notificación a dirección.

Criterios de logro

Condición	Criterio
Apto	El estudiante presenta un programa de consola completo (entrada validada, control de flujo, colecciones, funciones, menú en memoria) sobre el dominio de la verdulería. El código está en GitHub con README. Durante la defensa explica el funcionamiento general, cómo valida los datos y al menos una función en detalle.
No apto	El estudiante no completa alguno de los componentes del camino mínimo, no puede sostener una explicación coherente de su propio código, o no entregó el programa en GitHub. Recibe el detalle de objetivos pendientes y la comunicación formal del resultado.